

The XYZ of Theorizing in Software Engineering Research

KLAAS-JAN STOL, University College Cork, Ireland

A mature research discipline demonstrates cumulative tradition and a significant role for theory in its literature. Software engineering has developed sophisticated empirical traditions, but theoretical contributions often remain implicit and are not accumulated as deliberately as empirical findings. In this article, we present an argument that software engineering research does, in fact, “theorize”, but that these efforts are not sufficiently recognized, and thus are not present in the community’s shared awareness. The article presents a novel framework that positions eight general theorizing operations synthesized from prior literature on theorizing, that SE researchers use to make theoretical contributions. The purpose of this article is to name and organize these operations, and to illustrate how they are presently used in SE research. The framework offers a vocabulary that researchers can use to increase their awareness of what it means to make a theoretical contribution, and ultimately to stimulate a stronger focus on doing so. We contend that an increased awareness can help in stimulating this type of contribution more generally, which may help the SE community to mature by building a cumulative research tradition.

CCS Concepts: • **General and reference** → **Empirical studies; General literature; Surveys and overviews.**

Additional Key Words and Phrases: Theory development, theorizing, guidelines

ACM Reference Format:

Klaas-Jan Stol. 2026. The XYZ of Theorizing in Software Engineering Research. *ACM Trans. Softw. Eng. Methodol.* 1, 1 (October 2026), 38 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

*“Theory cannot be improved until we improve the theorizing process,
and we cannot improve the theorizing process until we describe it more
explicitly, operate it more self-consciously”*

— Karl E. Weick [128]

1 Introduction

Software engineering research has evolved considerably since its early beginnings in the sixties. Whereas earlier research primarily focused on the development of tools, techniques, and practices to design, develop and manage software systems, the nineties saw the emergence of experimentation. This, in turn, led to an increased recognition that the field should generate empirical evidence for the claims it made [62]. In the years that followed, the field has seen an impressive maturation and sophistication in its range and depth of empirical research methods [93, 99, 104, 116, 118, 135].

While the field has made very impressive advances in research approaches and techniques [31, 71, 104], a major blind spot that is becoming increasingly clear is the lack of attention to theory. While there have been some calls for increased attention to theory in the past two decades or so [53, 59, 115], theory development has by and large seen little uptake, and remains a theme

Author’s Contact Information: [Klaas-Jan Stol](mailto:k.stol@ucc.ie), k.stol@ucc.ie, University College Cork, Cork, Cork, Ireland.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7392/2026/10-ART

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

that does not appear to fit the way research is being conducted within the software engineering field. Consequently, many research results remain ‘localized,’ disconnected, and specific to the technologies of the day [112]. While the field certainly exhibits clear traces of cumulative research, much research in the field is driven by new phenomena and technologies without clear linkages to previous work.

This critique is not unique to software engineering; many scientific disciplines have struggled with this issue. In a scathing reflection on the state of ‘comparative politics,’ a subfield of political science, Sartori wrote in 1970 [98, p. 1039]:

“[...] much of our research is trivial and wasteful. [...] Graduate students are being sent all over the world [...] on ‘indiscriminate fishing expeditions for data.’ [...] The researcher sets out with a ‘checklist’ which is, at best, an imperfect net of his own. This may be an expedient way of handling his private research problems, but remains a very inconvenient strategy from the angle of the additivity and the comparability of his findings. As a result, the joint enterprise of comparative politics is menaced by a growing potpourri of disparate, non-cumulative and—in the aggregate—misleading morass of information.”

Similarly, Bourgeois [15, p. 443] lamented the state of research in organization and administrative science in 1979:

“[...] empiricism practiced without middle-range theory building as a goal carries the risk of running rampant. The result can be empirical studies that culminate in mere description, that count things for the sake of counting, or that generate data primarily for the purpose of applying rigorous statistical techniques.”

Theory plays an important role in making such cumulative research possible [34]. It allows findings from individual studies to be related to more general phenomena, provides explanations for why observed patterns occur, identifies conditions under which claims should hold, and connects otherwise separate bodies of knowledge. In this sense, theory provides structure to scientific knowledge, without which findings can remain tied to the particular technologies, systems, populations, or settings in which they were produced. This is particularly problematic in software engineering, where technologies and practices can change substantially over relatively short periods [41, 84]. A finding about a particular technology may have a relatively short lifetime, but insights into a more general phenomenon, of which a given technology is one instance, may have the potential to last longer.

Yet theory is often absent from much software engineering research. Researchers developing and studying techniques, practices, and tools may not consider themselves as doing “theory research.” To some, theory may appear as less relevant, or implying a particular research tradition that involves explicitly naming a theory, either to develop it or to test it. For example, several scholars within SE have explicitly proposed theories [9, 58, 89, 136].¹ These articles present theories as a final product, commonly featuring concepts and relationships among them presented in a ‘box-line’ diagram. It is easy to see how many studies do not fit that mold: they do not necessarily feature concepts or relationships between them. The suggestion that SE needs more theory can easily be misinterpreted as needing more box-line diagrams representing concepts and relationships. This article, instead, argues that such a focus on theory as a product is currently not helpful, but that it may be more productive to attempt to recognize the theorizing as a process that occurs in the software engineering literature. For that reason, we do not discuss or offer any definition of ‘theory.’

The main premise in this article is that much work in software engineering research already engages in theorizing, but that the community has not recognized these efforts as such, primarily due to a lack of awareness and vocabulary. This article seeks to correct this by (a) demonstrating how software engineering studies engage in theorizing, and (b) offering a vocabulary that allows

¹We draw a distinction between ‘scholars’ and ‘researchers,’ in an aspirational, not degrading, way. Researchers are those who conduct research studies; scholars are those who demonstrate scholarship by contributing theory which may have a more lasting impact.

researchers to describe theoretical advances. Consider, for example, Parnas's seminal work on modular decomposition [82]. Parnas's contribution is commonly understood as a design principle: modularization should be informed by information-hiding decisions rather than by the processing steps in a system, mirroring the structure of a flowchart. Parnas's work challenges the commonly accepted approach to organizing systems, and argues why the proposed alternative is better. Parnas's lasting contribution lies in an alternative way to think about modularity in a way that had not been considered before; as such, he contributed to theory about modularity, and yet, Parnas never invoked the term 'theory,' 'theorizing,' or other related term.²

If theorizing is left implicit and not articulated explicitly, other researchers may not recognize theoretical contributions, and thus may miss opportunities to build on those contributions. For a cumulative science to develop, papers must do more than report a set of findings. Subsequent work must be able to build on, transform, refine, and sometimes refute what earlier work has contributed. Theoretical products, such as concepts, propositions, and frameworks [44, 111], provide objects around which such cumulative work can develop, while *theorizing* provides the *operations* through which they change. If these operations remain implicit, theoretical development is more likely to remain fragmented across papers and subfields. However, if they can be recognized, they can also be pursued more deliberately. This article seeks to contribute to the goal of making the theorizing that happens in software engineering research more recognizable.

Several accounts of theorizing already identify activities involved in developing theory, including conceptualization, abstraction, integration, explanation, contextualization, contrasting, and problematization [2, 32, 68, 122] (Sec. 2). In this article we synthesize a set of recurring theorizing operations into a novel framework (Sec. 3), which we call the XYZ framework, mirroring the ABC framework [116].³ Each operation effects a theoretical 'transformation'; that is, an operation represents a change in the theoretical knowledge of a given topic that allows researchers to view the topic in a new light. Rather than suggesting a sequence of theory development steps, the framework offers an operational language that researchers can use to articulate theoretical contributions. A single study may perform several operations. The framework can help researchers recognize what theoretical work a study performs and, consequently, what kinds of theoretical work might follow from it.

We illustrate the eight operations using examples from both the history of science and from software engineering. These examples show that the eight operations are, in fact, widespread throughout the software engineering literature. Recognizing these contributions as theorizing allows them to be understood not only as individual results but also as parts in a continuing theoretical conversation [112]. Following the presentation of the framework, we present a detailed showcase of theorizing in the area of software testing, which remains a key topic within software engineering research (Sec. 4). We conclude the article with a discussion of implications, limitations, and an outlook on the future (Sec. 5).

2 Background

2.1 Theory development, theorizing strategies, and theorizing operations

Discussions of theory often focus on the main components of a theory, and the properties that a finished theory should have, including concepts, relationships between those concepts, and

²Many other papers in the software engineering field have become classics, including work by Lehman [64], Chidamber and Kemerer [21], and Potdar and Shihab [88], to name only a few. We contend that all such seminal papers share the characteristic that they exhibit traces of theorizing, even if they do not claim so explicitly.

³Similar to the ABC framework, where A, B, and C represent specific points on the ABC circumplex, the XYZ framework also identifies three points, which we labeled, not surprisingly, X, Y, and Z, representing Extension, Synthesis, and Realization, respectively. We explain these three points later on.

boundary conditions [3, 27, 115, 132]. Such descriptions are useful to understand what a theory might look like, but are of limited use in understanding how those products come into existence. Given that much software engineering research does not typically identify concepts and relationships amongst them, software engineering researchers may therefore conclude that such a focus is not relevant to them. We argue that it is this focus on theory as a product that is counter-productive. Instead, we contend that it is more useful to focus on the process of theorizing, and take the optimistic stance that software engineering researchers do, in fact, engage in theorizing.

It is useful to draw a distinction between *theorizing products*, *theorizing modes* or *strategies*, and *theorizing operations*. A theoretical product is a conceptual output from the theorizing process; Hassan et al. identified 12 such products [44], including ‘concept,’ ‘proposition,’ ‘framework,’ and ‘law.’ A theorizing product need not constitute a complete theory; instead it may be of help during further theorizing. Hassan et al. suggested that such intermediate products can be parts of a theory, or may be used to *frame* or *generate* new theory. However, by themselves these intermediate products are not theories.

A theorizing strategy, then, is the means or procedure by which a researcher is theorizing. Well-known strategies are grounded theory [40, 46], problematization [2], typology or taxonomy construction [90], hypothesis generation, or induction, abduction or deduction. Numerous scholars have identified strategies for theorizing; for example, Weick [128] reviews several strategies. The presentation of these strategies varies from editorial comments [129, 132], to academic articles [87, 128], to book-length treatises [27, 103, 122].

At the lowest level sit what we call theorizing operations, which is the focus of this article.⁴ These describe the elementary transformations of a theoretical understanding. Through these operations, a scholar changes what is represented, what is related, explained, or delimited. Theorizing strategies tend to use one or more such operations, and therefore sit at a higher level of abstraction.

Table 1. Theorizing products, strategies, and operations

Level	Description	Utility to scholars	Example
Theorizing Products	Intermediate theorizing products that are not developed into complete theories, but represent an outcome towards that ultimate goal.	Assists scholars in recognizing the value of various intermediate theorizing products and their role in the theorizing process.	Hassan et al. identify 12 such products, including framework, statement, law, concept, construct, metaphor, analog [44].
Theorizing Tactics and Strategies	An approach to theory development that describes a series of steps or a process that has as potential outcome a theory.	Offers a procedure to follow to develop theory.	Grounded Theory [40], inductive reasoning from case studies [28], hypothesizing [70], middle-range theorizing [15].
Theorizing Operations	The elementary operations that add a new layer of understanding to an emergent phenomenon or theory. Each operation ‘transforms’ insights, and makes a theoretical contribution to an emergent, but frequently incomplete and implicit theory.	Helps scholars to identify ways in which a study makes a theoretical contribution, and to design new studies that can do so.	Naming or conceptualizing a phenomenon, distinguishing or refining phenomena or concepts, or problematizing assumptions [2].

⁴We focus on ‘operations,’ analogous to the operations (or instructions) of a Reduced Instruction Set Computer (RISC); that is, these are the basic operations that one can perform. A theorizing *strategy* could then be seen as a ‘program’ consisting of one or more operations.

2.2 Existing Guidelines on Theorizing

Numerous accounts are available that seek to explain how researchers can develop theory, both within and beyond the software engineering discipline. Table 2 provides only a small selection of these.

Table 2. Selected guidelines for theorizing across several disciplines

Source	Field	Main contribution
Dubin 1978 [27]	Sociology	Discusses theory development by identifying units, laws of interaction, boundaries, system states, propositions, empirical indicators, and hypotheses.
Poole and van de Ven 1989 [87]	Management	Provides four strategies for leveraging theoretical tensions and contradictions to build theories, as an alternative to theories that seek to unify observations.
Bacharach 1989 [3]	Management	Identifies components of a theory. Comments on criteria for evaluation of theories, and distinguishes theory from those products that are not theory.
Weick 1995 [129]	Management	Comments on theorizing as a process, and on intermediate products of theorizing, including references, data, variables, and diagrams.
McGuire 1997 [70]	Psychology	Describes 49 heuristics for generating hypotheses across 5 categories.
Wacker 1998 [127]	Operations management	Presents a general procedure for theory building, discusses different approaches to theory building, including an analysis of their use in operations management.
Shoemaker et al. 2004 [103]	Communication and media	Offers a step-by-step guide for building social science theory, while acknowledging the iterative nature of theory development.
Sjøberg et al. 2008 [105]	Software engineering	Proposes a framework for describing software engineering theories and offers advice on how to propose, test, modify, and use theories.
MacInnis 2011 [68]	Marketing	Distinguishes conceptual contributions including identifying, delineating, differentiating, integrating, revising, and refuting.
Alvesson and Sandberg 2011 [2]	Management	Develops research questions by identifying and challenging assumptions underlying established theories and literatures.
Tsang and Ellsaesser 2011 [125]	Management	Discusses ‘contrastive explanations’ as a technique to support theory building, and offers eight heuristics to apply this technique.
Höök and Löwgren 2012 [49]	Human-computer interaction	Introduces ‘strong concepts’ as an intermediate form of design knowledge, that sits between generalized theories, and knowledge of a particular instance, focused on interaction design research.
Swedberg 2014 [122]	Sociology	Provides practical tools including observation, naming, concept formation, analogy, metaphor, typology, and explanation.
Wieringa and Daneva 2015 [134]	Software engineering	Provides six strategies for developing theories of the middle range.
Burton-Jones et al. 2015 [18]	Information systems	Defines theoretical perspectives through choices of concepts and relationships and introduces conceptual latitude and conceptual fit.
Fisher and Aguinis 2017 [32]	Management	Offers tactics including construct specification, construct splitting, contrasting, and alternative ways of structuring theoretical relationships.
Busse et al. 2017 [19]	Management	Systematizes the meaning, identification, exploration, and use of boundary conditions in theoretical research.

Continued on next page

Table 2 *continued from previous page*

Source	Field	Main contribution
Shepherd and Sud-daby 2017 [102]	Management	Integrates a large literature on theorizing tools around conflict, character, setting, sequence, and plot and arc.
Ralph 2018 [90]	Software engineering	Offers guidelines for developing process theories and taxonomies in software engineering.
Jaakkola 2020 [51]	Marketing	Distinguishes theory synthesis, theory adaptation, typology development, and model construction as approaches to conceptual research.
Borsboom et al. 2021 [10]	Psychology	Provides a methodology for constructing explanatory theories, comprising five steps.
Beaudouin-Lafon et al. 2021 [6]	Human-computer interaction	Introduces the concept of Generative Theories of Interaction as a theory building framework, illustrated with three examples.
Rivard 2021 [91]	Information systems	Proposes an iterative spiral model for theory building plus a series of design principles for building good theory.
Leidner and Tona 2021 [65]	Information systems	Offers a ‘confessional account’ of the authors’ experience, explaining activities and phases in theory development, captured in a model which includes four theorizing processes.
Hoda 2021 [46]	Software engineering	Proposes socio-technical grounded theory; a variant of GT specifically developed for socio-technical fields including software engineering.
Breslin and Gatrell 2023 [16]	Management	Identifies eight strategies across the ‘miner-pro prospector’ continuum, contrasting low risk/return (miners), vs. high risk/return (prospector).
Ortiz de Guinea and Watson 2024 [80]	Information systems	Discusses techniques to develop theory from prior literature.

Most proposed procedures for theory development consist of a number of steps. A classic representative work is by Dubin [27] who discusses a series of steps: (1) identify units (concepts), (2) establish laws of interactions (relationships among units), (3) establish boundaries, (4) establish system states, (5) establish propositions (logically derived statements), (6) identify empirical indicators, (7) derive hypotheses from propositions, and (8) testing. Another similar, but simpler procedure is offered by Wacker: (1) define variables, (2) limit the domain, (3) establish relationships, (4) make predictions and provide empirical support. Perhaps a key reason why such procedures are less helpful for budding software engineering researchers is that such a mechanistic and formulaic template is a poor fit for our field. To illustrate this poor fit, consider the topic of generative AI in software engineering, which at the time of writing has attracted considerable attention. Much of the research has started to explore this emerging phenomenon by asking questions how it is being used in practice. For example, researchers have investigated how developers use AI coding assistants across different development activities through large-scale surveys [100], while others have identified open questions across software engineering rather than testing an established theory [78]. Other work has explored how GenAI may affect creativity in software development [52]. In these examples, researchers are simultaneously exploring the phenomenon, identifying relevant concepts and relationships, and developing questions for future research. While one could apply procedures such as those proposed by Dubin and Wacker, in practice this does not happen.

Two works in Table 2 are closely relevant to our work. MacInnis’s framework for conceptual contributions [68] identifies four key conceptual goals: envisioning, explicating, relating, and

debating. Each of these goals has two specific goals that are positioned as opposites. While MacInnis presents these in a circular diagram, the axes of the diagram have no further meaning. Further, while several of the theorizing operations that we identify map well to MacInnis's activities, others do not. For example, while MacInnis identifies 'advocacy' and 'summarization,' we argue that neither are theorizing operations, because they do not effect a 'transformation' in the knowledge development process.

A second framework that is closely related is what Breslin and Gatrell [16] have called the Miner-Prospector continuum. At the root of their continuum lies the metaphor of miners vs. prospectors; miners are those workers with relatively low, but reliable payback. Prospectors, on the other hand, face a much higher risk but also a much higher return on investment. They identify 8 strategies across the miner-prospector continuum that capture the theoretical "return on investment."

A common theme across most guidance for theorizing, including those listed in Table 2, is that these guidelines are *generative*. That is to say, virtually all guidance seeks to provide advice to readers on how to produce theory. In this article, we do not propose or advocate any specific method to develop theories. As the SE field is maturing, it must find its own way to articulate theories, rather than following general guidance that suggests the identification of concepts and relationships among them. Some scholars in the SE field have proposed methods tailored to SE. Sjøberg et al. proposed a template for SE theories comprising actors, technology, activity, and software system as 'archetype' concepts [105], and a set of five steps to develop theories. Hoda proposed socio-technical grounded theory as a variant of the classic methodology that may be more accessible to SE researchers [46].

While we applaud these proposals, this article pursues a different and more modest ambition. Rather than suggesting a new approach to theory development, we seek to *recognize* what the field is already doing, and through the lens of the proposed framework, we argue that much work that is being done, can, in fact, be classified as forms of "theorizing."

3 The XYZ Framework for Theorizing Operations

This section presents the XYZ framework that lays out eight theorizing operations (see Figure 1). A theorizing operation can be performed by a researcher to theorize, that is, to make a theoretical contribution. These operations can be characterized as theorizing 'primitives': these are the fundamental actions a researcher can take, and typically a research study would report on one or more of these operations. The framework is the result of a synthesis of guidelines on theorizing in other disciplines, as well as reflections on the author's experiences on his own theorizing research over the past decade or so [115, 117, 119, 124, 130]. Appendix A presents details on the synthesis.

In the remainder of this section, each operation is presented and illustrated with general science examples as well as software engineering examples. It is important to note that studies may perform multiple operations, or even follow a theorizing strategy (see Table 1). For each example, we focus specifically on one operation to illustrate what this looks like in SE research. Operations also address one of four theoretical shortcomings, represented by the four quadrants (see Table 3). Quadrant I represents conceptual ambiguity, suggesting a metaphorical 'fog' is cast over our understanding. Quadrant II represents limited theoretical depth; our understanding is like 'shallow water.' Quadrant III represents relational incompleteness, suggesting there are 'loose threads' that remain to be tied up. Finally, Quadrant IV represents contextual inadequacy; we must 'stress test' a theory or concept in the real world, and adjust if needed. After the eight operations are presented, we discuss their positioning and the internal structure of the framework to provide further vocabulary that can be used to clarify the theorizing process, as well as the X, Y, and Z markers.

Table 3. Quadrants of the XYZ framework

Quadrant	Theoretical shortcoming	Description	Metaphor
I	Conceptual ambiguity	The phenomenon is not clearly defined or differentiated.	Fog
II	Limited theoretical depth	Understanding remains close to observation, with limited abstraction or explanation.	Shallow water
III	Relational incompleteness	Relevant relationships or conditions of applicability remain unspecified.	Loose threads
IV	Contextual inadequacy	The adequacy of existing theoretical ideas in a particular empirical context remains unresolved.	Stress test

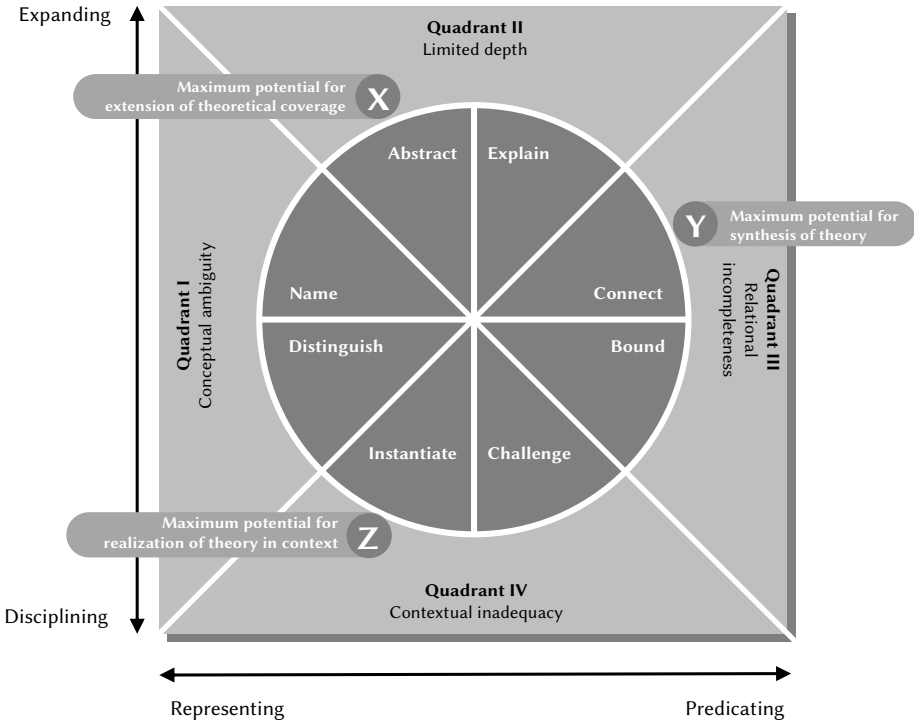


Fig. 1. The XYZ Framework: eight theorizing operations. Quadrants I to IV represent four theoretical shortcomings addressed by the operations within each quadrant.

3.1 Name

As scholars encounter phenomena that appear worthy of study, a key theoretical operation is to name them. Naming provides a common term through which a phenomenon can be discussed, compared, and subjected to further study. Naming something addresses conceptual ambiguity (Quadrant I). A name does not constitute an explanation for the phenomenon, nor does it necessarily provide a complete definition, nor does it require evidence that the phenomenon exists: it may be hypothetical. Hempel wrote [45, p. 47, cited by Sartori [98]]:

“it is precisely the discovery of concepts with theoretical import which advances scientific understanding; and such discovery requires scientific inventiveness and cannot be replaced by the—certainly indispensable, but also definitely insufficient—operationist or empiricist requirement of empirical import alone.”

Hempel’s remark is relevant to SE research, which excels at measuring given the widespread available traces in the form of developer activity (e.g. commits, defects). However, simply naming these empirically measurable entities does not demonstrate the theoretical importance of these entities. Nor is it straightforward to name things; a classic example of this is the conflation of lines of code produced with the concept ‘productivity.’

An example in the history of science is the introduction of the term ‘gene.’ In 1909, Wilhelm Johannsen introduced this term for the hereditary units discussed in Mendelian genetics [56]. At that time, the physical nature of the gene was not yet known; there was no knowledge of the physical (molecular) nature of genes, but the experiments by Mendel had provided evidence for the existence of hereditary factors. Giving the hereditary unit a stable name made it possible to formulate and organize subsequent questions about its location, transmission, effects, and physical nature.

Naming is a theorizing operation previously recognized by scholars in other disciplines, including social science and management. Swedberg explicitly identifies naming as an early activity in theorizing, writing: “To give something a new name essentially means to give it a new identity” [122, p. 56]. Swedberg argues that not naming a phenomenon may risk that it “will slip through your hands”; in other words, if a certain phenomenon is not assigned its own label, it will not receive attention for further study, and it may lose focus in further inquiry. Glaser explicitly describes conceptualization in grounded theory as involving the naming of recurring empirical patterns [39, p. 25]. It is worth noting that Swedberg distinguishes between naming a thing and developing it into a concept. In this article, we do not distinguish between naming and conceptualizing;⁵ instead, in our view, both labeling a phenomenon by giving it a name, and the formal definition of a phenomenon including identifying its attributes are forms of naming. MacInnis refers to *identifying* as a form of conceptual contribution through which an entity that was previously unrecognized becomes available for further theorizing [68]. Although these authors use different terminology, they describe a common activity: a previously unidentified, ambiguous, or inconsistently described phenomenon is given an explicit label by which it can be referred to.

A well known example in software engineering is the term ‘technical debt.’ Cunningham introduced the debt metaphor in 1992 [24]. Developers had long made design decisions that enabled short-term progress but created additional work later. The term technical debt, however, provided a concise way of referring to this phenomenon. The financial metaphor imported a wider vocabulary through which the phenomenon could be reasoned about: debt may be incurred, repaid, and may accumulate interest. The coining of the term technical debt also demonstrates the generative value of naming as a theoretical operation. Once it had been identified as a distinct object of inquiry, researchers started to ask many new questions that had hitherto not been considered, leading to a workshop series since 2010, and a dedicated conference series since 2018.

3.2 Abstract

Scholars may observe different phenomena and realize that these are, in fact, different instances of a single higher-level phenomenon. This operation involves moving away from the particulars of an area and recognizing it as an instance of a higher level phenomenon. Doing so makes it possible to reason about several previously separate phenomena and ‘recast’ them using a common

⁵We suggest naming and conceptualizing lie on a continuum, and are not categorically different; some scholars may simply coin a name for a phenomenon, and another could do the same, but develop it in more depth as a concept with a specification of attributes.

vocabulary and, potentially, a common theoretical account. In other words, scholars may *abstract* phenomena into a more general phenomenon. Doing so addresses a limited theoretical depth of our understanding, which is why this operation is placed in Quadrant II.

A classic example is Newton's theory of gravitation. Newton treated the motion of an apple falling from a tree and the motion of the Moon orbiting the Earth as two different manifestations of the same higher-level phenomenon: motion governed by the same principles of gravitation [77]. The abstraction does not imply that an apple and the Moon are equivalent in all respects. Rather, it identifies a property of their motion that can be understood through the same higher-level account. The theoretical operation is therefore to unify several apparently distinct observations into a more general phenomenon that subsumes them.

Abstraction is an operation that can be commonly found in discussions on theorizing. In grounded theory, empirical incidents are compared and progressively grouped into abstract concepts and categories [39, 40]. Sartori describes a related process through his "ladder of abstraction," whereby concepts can be moved to higher levels of generality by increasing their extension across cases while reducing the number of attributes used to characterize them [98]. These authors use different terminology, but they describe a common intellectual operation: phenomena that were previously considered separately are represented as instances of a more general conceptual category.

A useful example in software engineering is the emergence of the concept of a software ecosystem. Several research literatures had previously studied phenomena that, while different in their immediate focus, shared several characteristics. One such literature is that on open source software (OSS), which focuses on understanding how large software systems could be developed by distributed communities whose members were not organized in the same way as conventional development teams. For example, Mockus et al.'s seminal work studied the Apache web server and Mozilla web browser, examining issues such as developer participation, ownership, productivity, defects, and coordination [74]. Apart from studying how OSS works, there is also considerable research on how organizations use OSS products as components [50, 108].

A second stream of literature focuses on software product lines (SPL). An SPL consists of a family of related products that share a set of assets while differing in functionality required by individual products [86]. A central concern is the management of commonality and variability across a family of systems. SPL research shifted the unit of analysis from a single software product to a family of related products and the reusable assets from which they are derived. A substantial body of work focused on the technical problem of representing and realizing variability, including feature models, variation points, configuration mechanisms, reusable architectures, and product derivation [14, 23, 61, 86, 121]. SPL engineering also required changes to the development process itself. A common distinction is between *domain engineering*, which develops reusable components, and *application engineering*, which derives individual products from those assets [23, 60, 86]. Finally, adoption of an SPL may require substantial organizational change, including decisions about the division of responsibilities between platform and product teams, the ownership of shared assets, and the organizational units responsible for domain and application engineering [1, 11, 12].

A third relevant literature focuses on software supply networks. Earlier work had already begun to question the assumption that software products were developed and delivered by isolated organizations. Farbey and Finkelstein proposed a research agenda around software supply-chain business architectures [29]. Jansen et al. subsequently argued that software firms increasingly operate in networks of interdependent participants involved in software design, development, implementation, publication, and services [54]. Their work proposed explicit representations of such networks in order to reason about dependencies, responsibilities, liabilities, and business opportunities. Here, the unit of analysis extends beyond the *individual* software organization to a *network* of technically and economically interdependent organizations.

A fourth literature relevant to this example focuses on the business of software. For example, Cusumano examined how software firms compete not only through individual products, but also through combinations of products, services, partnerships, and complementary offerings [25]. This work similarly shifted attention away from a stand-alone software product toward a wider network of actors on which its development, distribution, and commercial success depends.

These four literatures used different terminology and focused on different focal problems. Open source research studied distributed communities; software product line research studied families of related products and shared assets; software supply-network research studied inter-organizational dependencies; and software-business research studied platforms, partners, markets, and complements. The concept of a *software ecosystem* provides a higher-level abstraction through which these phenomena can be considered together. Early work on software ecosystems explicitly drew attention to the interaction of software platforms, organizations, developers, complementors, and markets. Bosch, for example, described a transition from software product lines toward software ecosystems as platforms and reusable assets extended beyond the boundaries of a single organization [13]. Jansen et al. similarly characterized software ecosystems in terms of interacting actors, a shared market for software and services, and the relationships among those actors [55].

The theoretical contribution of the abstraction operation is to explicitly link seemingly distinct literatures by recognizing a common set of concerns; the foundation for this operation is the recognition that what appear to be different things are, in fact, instances of a common category. In other words, OSS communities can be seen as an *instance* of a software ecosystem, as they supply reusable components, and companies adopting those components become dependent on those communities as suppliers. That same supplier/customer relationship can also be found within organizations that adopt an SPL architecture. The abstraction allows us to take findings from one literature and leverage those to the higher-level concept of software ecosystems. At this higher level concept, questions originating in one literature may become meaningful in another. For example, researchers can ask how an ecosystem is governed, how actors enter and leave it, how technical dependencies interact with organizational relationships, and how the health or sustainability of an ecosystem can be assessed.

Abstracting is likely to involve narrowing focus on some details that may have been important in the original literature. For example, while variability management techniques are key to the technical SPL literature, within the wider software ecosystem literature, such details become less relevant. Such a shift in focus is inherent to abstraction in general. The purpose of the operation is to identify a level of representation at which previously separate observations can be treated as instances of the same phenomenon. This may broaden the relevance of findings from one literature to other fields, and open up new research questions.

3.3 Explain

Scholars may investigate how two phenomena or concepts are related in order to explain that relationship. This operation further expands our knowledge of a given phenomenon. The explanation that is developed may identify a mechanism, process, or underlying logic that connects concepts or observations. Explaining goes beyond observing a regularity or identifying an association: it proposes *why* that regularity should occur. Explain adds further theoretical depth, which is why it is placed in Quadrant II.

A historical example is scurvy [5]. By the eighteenth century, it had become increasingly clear that certain foods could prevent or cure the disease. James Lind's well-known experiment in 1747 provided evidence that sailors suffering from scurvy benefited from citrus fruit [5]. This was an important empirical observation, but it did not explain *why* citrus had this effect. The explanation emerged much later, when scurvy came to be understood as a consequence of insufficient vitamin

C. The theorizing step was therefore to progress from the observation that citrus helped, to an account of *why* it helped. Once the relevant mechanism was understood, the explanation was no longer tied to citrus itself: other foods containing sufficient vitamin C could be expected to have the same effect.

Explaining refers to the scholar's activity involving the development of an explanatory account. Providing an explanation does not imply that the proposed explanation is correct, nor that it has already been empirically tested. Consequently, theorizing may produce explanations that later turn out to be wrong. The history of science provides many examples. Bloodletting, for instance, was practiced for centuries, and was based on a theory that health depended on a balance of bodily fluids and that disease could result from an imbalance, such as an excess of blood [81]. While clearly incorrect, the theory provided an explanation for why illness occurred and why removing blood might offer a cure.

The distinction between correlation (e.g., scurvy and citrus fruit) and explanation (e.g., a lack of vitamin C) is also reflected in the literature on theorizing. Whetten distinguishes the 'what' and 'how' of a theory from its 'why': specifying the relevant concepts and relationships is insufficient without a theoretical rationale for why those relationships should hold [132]. Swedberg similarly treats the construction of an explanation as a central activity of theorizing [122]. Explanations can also take different forms. In their presentation of process theories, Van de Ven and Poole, for example, identify several different theoretical "motors" for explaining development and change, including life-cycle, teleological, dialectical, and evolutionary process theories [126] (see also Ralph [90]). These types of process theories can offer different ways to explain a phenomenon.

An example of this theorizing operation in software engineering can be found in the grounded theory study by Leite et al. on organizational structures for development and infrastructure professionals [66]. Organizations structure the relationship between software development and infrastructure work in different ways. These include, for example, arrangements in which development and infrastructure are organized in separate departments, arrangements in which the two collaborate closely, and arrangements in which they form a single organizational unit. Merely identifying such structures is descriptive. Leite et al. sought to go further by asking *why* organizations adopt different structures, and developed an account that relates organizational conditions and causes to the structures that organizations adopt, as well as to their consequences and contingencies. The theoretical contribution therefore lies not only in identifying different organizational structures, but in proposing an explanation of why particular structures arise under particular conditions and what follows from them. In doing so, the study not only asks "what organizational structures exist?" but more interestingly "why do organizations adopt these structures?"

The operation of explaining expands our knowledge of an observed phenomenon. Such an account may subsequently be tested, refined, bounded, connected to other explanations, or challenged; each of these is a separate theorizing operation.

3.4 Connect

Scholars may advance a field of inquiry by connecting concepts, theories, or bodies of knowledge from two hitherto separate research areas. By doing so, a scholar can leverage knowledge and insights from one field to advance or inform another field. Such a connection does not imply that the two phenomena are instances of the same higher-level concept, as is the case for the 'abstract' operation. Rather, it identifies a useful relationship between distinct conceptual systems, allowing knowledge developed in one area to inform another. Connect addresses relational incompleteness, which is why it is placed in Quadrant III.

A historical example is Claude Shannon's use of Boolean algebra to reason about electrical switching circuits [101]. George Boole, the first Professor of Mathematics at Queen's College

in Cork, Ireland (now University College Cork), developed what he called “an algebra of logic” [8], which later came to be known as Boolean algebra. Several decades later, Shannon realized that Boolean algebra could be used to represent, analyze, and simplify switching circuits [101]. Shannon’s contribution lies in the connection between two previously distinct domains. Electrical engineers already knew how to construct switching circuits, while Boolean algebra already provided a formal system with established operators and laws. By connecting the two, Shannon made the existing body of Boolean reasoning available for the analysis and design of circuits.

Connecting different domains of knowledge as a form of theorizing can be observed in other guidelines for theory development. MacInnis describes ‘integrating’ as a form of conceptual contribution in which previously separate conceptual entities are brought together [68]. Fisher and Aguinis similarly discuss theory elaboration through the restructuring of relations among constructs [32]. Jaakkola uses the term ‘theory synthesis’ for conceptual work that achieves integration across theories or streams of literature that have previously been treated separately [51]. In his discussion of ‘connectivity’ as a quality of theories, Bacharach wrote [3, p. 511]:

“Connectivity refers to the ability of a new theory to bridge the gap between two or more different theories, thus explaining something between the domains of previous theories. In this way, new knowledge is created, and a more nearly continuous mapping of the empirical universe is achieved.”

Indeed, Shannon’s linkage between Boolean algebra and electrical circuit design is one such example of ‘bridging’ that led to “new knowledge.”

An example in software engineering is the use of control theory in research on self-adaptive software systems. Self-adaptive systems are expected to monitor their operation and modify their behavior in response to changes in their environment or requirements. Software engineering approaches often realize such adaptation through feedback loops and express desired behavior using requirements or formal properties. Control theory, however, has developed a substantial body of knowledge for reasoning about feedback systems, including concepts such as stability, settling time, steady-state error, and robustness. Caldas et al. explicitly connected these two bodies of knowledge by examining how properties used in control theory relate to properties used in software engineering for self-adaptive systems [20]. Their starting point was that both communities reason about feedback-controlled systems, but use different concepts and representations. The authors therefore developed a mapping between control-theoretic properties and software engineering properties, using specification patterns as a common language.

The theoretical contribution lies in identifying correspondences that allow concepts and results from one field to be used in the other. Once the connection is made, existing questions can be reformulated, or new questions may arise from the newly acquired perspective. Instead of asking only whether an adaptive system satisfies a software requirement, researchers can also draw on control systems terminology and ask whether its adaptation mechanism is stable, how quickly it settles after a disturbance, or whether guarantees established at the controller level survive implementation in software.

Connecting is a separate operation from abstracting. Abstracting brings different observations together because they can be treated as instances of the *same higher-level phenomenon*. Connect, in contrast, retains the distinct identities of the concepts or bodies of knowledge involved, but establishes a relationship between them so as to allow leveraging insights from one field in another. That is, knowledge developed for one conceptual system can become useful for reasoning about another without requiring the two to be unified into a single phenomenon.

3.5 Bound

Scholars may consider an existing explanation or a relationship that hitherto has been assumed to hold generally, and study its boundary conditions. That is, a given relationship may only hold

under some conditions but not others. We label this theoretical operation ‘bounding.’ Through this theoretical operation, a scholar *qualifies* previously identified results or assumptions within a field of research. Whereas Connect addresses relations to other theories and concepts, Bound addresses how a theoretical claim relates to the conditions and contexts that delimit its applicability, which is why it is placed in Quadrant III.

A historical example is Newtonian mechanics [110]. Newton’s laws provide highly accurate predictions for a large range of physical phenomena. However, Newtonian mechanics cannot provide sufficiently accurate predictions at very high speeds or in very strong gravitational fields; under those conditions, one needs Einstein’s theory of relativity. Further, Newtonian mechanics also does not provide sufficiently precise predictions at atomic and subatomic scale; in such cases, quantum mechanics is required. Thus, the theoretical advances by Einstein and quantum mechanics provided a qualification of Newtonian mechanics; the latter theory became less universal.

Bounding is well established in the literature on theory development. Whetten argues that theoretical contributions should specify not only *what*, *how*, and *why*, but also *who*, *where*, and *when* [132]. Dubin, Bacharach, and Busse treat boundary assumptions as an explicit part of theory [3, 19, 27]. Johns shows how contextual conditions can alter both the meaning of phenomena and the relationships among variables [57]. In the specific case of statistical models, one could think of bounding as adding one or more moderators: given a relationship between X and Y, a moderator W adjusts the strength of the relationship, or the range of values for which the relationship holds. Across these accounts, the common theoretical operation is to replace a presumed general claim with a more conditional one.

A good example of bounding research in software engineering is research on pair programming. Early discussions of pair programming often focused on whether working in pairs improves outcomes compared to working individually. Hannay et al.’s meta-analysis of pair-programming studies [43] showed that the effects were not uniform. Pair programming could reduce completion time for relatively simple tasks, whereas for more complex tasks it was more likely to improve solution quality, although often at the cost of greater effort. The theoretical contribution of bounding is therefore to qualify under which circumstances (e.g., tasks, developer experience, etc.) pair programming is effective.

3.6 Challenge

Scholars may make a theoretical contribution by questioning assumptions within a field of inquiry. A theory, concept, or design principle may be widely accepted because it has become the conventional way of thinking about a problem. A further theoretical operation is therefore to *challenge* such an assumption, and ask whether an alternative account might be more appropriate. Challenge confronts an existing theoretical account with assumptions or evidence that does not fit the theory; thus, it addresses a ‘contextual inadequacy,’ which is why this operation sits in Quadrant IV.

A historical example is Rutherford’s challenge to the accepted model of the atom [48]. At the beginning of the twentieth century, Thomson’s “plum pudding” model was widely accepted [47], but Rutherford’s experiments produced results that did not fit well with this model. He therefore proposed a different account of the atom, in which most of its mass was concentrated in a very small central nucleus [94]. The key theoretical step was to recognize that the existing model could not explain the observations and to propose an alternative.

Challenging established assumptions is recognized explicitly in the literature on theorizing. Alvesson and Sandberg contrast conventional ‘gap spotting’ with *problematization*, in which researchers identify and question assumptions underlying existing theories and literatures [2]. Sandberg and Alvesson similarly argue that research questions can be generated by exposing and questioning such assumptions rather than merely identifying topics that have not yet been studied

[97]. Using an alternative term, MacInnis describes *refuting* as one form of conceptual contribution, whereby an accepted way of understanding a conceptual entity is challenged [68]. The common theme is that scholars question some fundamental assumptions or theory.

A classic example in software engineering is Parnas's work on modular decomposition, briefly discussed in Sec. 1. By the early 1970s, modularization was widely regarded as desirable because it could improve the comprehensibility, flexibility, and maintainability of large software systems. The conventional approach was to decompose a system according to the major steps in its processing or according to the tasks performed by the system, effectively following the structure of a flowchart. Parnas suggested this might not be the best way to decompose a system, and instead offered an alternative criterion, namely, information hiding [82]. Parnas illustrated the argument by presenting two different decompositions of the same system. One followed the conventional processing-oriented approach; the other decomposed the system around design decisions that were likely to change. He showed that the latter decomposition had several advantages, because changes to particular design decisions could be confined to individual modules rather than propagated throughout the system. This led to the principle that modules should hide design decisions from one another, which subsequently became known as information hiding. Parnas thus made a theoretical contribution by challenging a previous assumption, proposing an alternative and demonstrating its advantages.

Challenging is different from bounding, because in the former, the fundamental theoretical claim is called into question or shown to be incorrect; the latter only qualifies and specifies the conditions under which a certain theoretical claim holds.

3.7 Instantiate

Scholars may take an existing theoretical account and ask what it means in a particular domain, context, or case. The theory itself need not be new. The theoretical contribution lies in working out how its concepts and relationships manifest themselves in the setting under study. We refer to this operation as *instantiate*. Instantiating a theory or concept moves from the general to the particular: it asks what a general concept or theory looks like in a specific setting that had hitherto not been studied. In that sense, instantiation seeks to resolve the contextual inadequacy, which is why it is placed in Quadrant IV.

A historical example is Archibald Garrod's work on alkaptonuria, which is a rare inherited metabolic disorder in which the body cannot break down certain amino acids [63, 85]. Following the rediscovery of Mendel's work around 1900, Mendelian inheritance provided a general account of how hereditary characteristics could be transmitted across generations. Garrod observed that the occurrence of alkaptonuria within families was consistent with Mendelian inheritance. In 1902, he argued that the condition behaved as a recessive inherited characteristic [38]. This was an important step because Mendel's theory had been developed from experiments with plants; Garrod asked whether the same inheritance principles would apply for human disease. The result was not a new theory of inheritance, but an instantiation of an existing theory in a new domain. Garrod's analysis of alkaptonuria is widely regarded as an early demonstration of Mendelian inheritance in human disease.

The use and application of theory to a particular context has been recognized by scholars as a form of theorizing. Whetten distinguishes *contextualizing theory*, whereby an existing theory is examined in a particular setting, from theorizing about context itself [133]. His discussion of cross-context theory borrowing is relevant here: bringing a theory into a new setting requires researchers to determine how its constructs and relationships should be interpreted in that setting. Gregor and Jones use the more specific notion of an *expository instantiation* in the context of design theory, where a concrete realization demonstrates what a more general design theory looks like in

practice [42]. The common theme across these reflections on theory is the movement from a more general theoretical structure toward a specific instance or context.

An example in software engineering can be found in the application of Social Representations Theory to code smell identification by De Mello et al. [26]. Social Representations Theory was developed in social psychology to explain how communities collectively construct and share understandings of socially relevant objects. Rather than treating knowledge solely as an individual cognitive property, the theory focuses on the values, beliefs, practices, and associations through which groups make sense of a particular object. Their question was not whether developers can identify code smells, but how software developers *collectively* represent the activity of smell identification. Using developers' associations with the task, they studied which ideas formed part of its social representation. Their results showed, for example, strong associations between smell identification and smell removal, bugs, code inspection, and the individual skills required to identify smells. They then compared these representations with the concerns raised in the research literature, identifying areas of misalignment between research and practice.

The theoretical contribution arises from asking what Social Representations Theory means when the “social object” is a software engineering activity. The theory provides a vocabulary and a set of expectations about what should be examined: not only whether an individual developer recognizes a smell, but which beliefs, practices, and associations are shared within a community, how these representations differ across groups, and whether the representation in research corresponds to that held by practitioners. The general theory therefore changes which aspects of the software engineering phenomenon become visible and which questions can be asked about it.

Instantiating should be distinguished from connecting. The connect operation establishes a relationship between distinct bodies of knowledge so as to leverage knowledge and insights from one domain in another. Instantiation starts with a more general theoretical account and works out what that account means in a particular setting.⁶

3.8 Distinguish

Scholars may make a theoretical contribution by arguing that a given phenomenon or concept has been treated ambiguously or has conflated distinct meanings, and by drawing a clearer distinction between them. A further theoretical operation is therefore to separate such concepts and make the distinction explicit. We refer to this operation as ‘distinguish.’ Distinguishing does not necessarily introduce a new phenomenon; rather, it zooms in on an existing phenomenon or concept, and increases the conceptual “resolution.” Because it addresses conceptual ambiguity (by separating concepts), it sits in Quadrant I.

A historical example is Joseph Black’s distinction between heat and temperature [36, 131]. Before Black’s work in the eighteenth century, heat and temperature were often treated as closely related aspects of the same phenomenon. Black argued that they should be separated. Two bodies can have the same temperature while differing in the amount of heat required to bring them to that temperature, and equal quantities of heat can produce different changes in temperature in different substances. This distinction led to the concepts of *heat capacity* and *latent heat*, and became important for the subsequent development of thermodynamics [7]. The theoretical advance was the recognition that a single conceptual category covered two different quantities.

Distinguishing is also recognized in several accounts of theorizing. MacInnis identifies *differen-*
tiating as a form of conceptual contribution in which entities that have previously been treated

⁶Abstract, connect, and instantiate are all three forms of linking; a useful though imperfect analogy familiar to software engineers to distinguish these three is the object-oriented mechanisms inheritance (abstract), association (connect), and implementation or realization (instantiate).

together are separated on theoretically meaningful grounds [68]. Fisher and Aguinis describe related operations such as horizontal and vertical contrasting and construct splitting, through which a construct is decomposed or distinguished from neighboring constructs [32]. Suddaby similarly argues that theoretical clarity depends on specifying what a construct is and how it differs from related constructs [120]. Sartori’s discussion of concept formation likewise emphasizes the importance of maintaining conceptual distinctions and avoiding concept stretching [98]. These accounts use different terminology, but share the idea that theoretical progress may arise by revealing differences that an existing conceptualization hides.

An example in software engineering concerns the concepts of code readability and code legibility. These terms, together with related terms such as understandability and comprehensibility, have often been used interchangeably. Oliveira et al. reviewed 54 human-centric studies and argued that readability and legibility should be treated as distinct, though related, properties [79]. They define readability in terms of structural and semantic characteristics that affect how easily developers can read and understand code. Legibility, in contrast, is concerned with visual characteristics that affect how easily developers can identify elements of the code, such as spacing and alignment.

Differentiating clearly between these concepts allows researchers to ask more specific questions. A formatting change may make identifiers or program elements easier to locate without making the program easier to understand. Likewise, meaningful identifiers or familiar programming constructs may improve comprehension without changing the visual presentation of the code. Treating both effects as “readability” conflates research findings and makes it difficult to determine what an empirical study has actually measured. Oliveira et al. introduced the separation explicitly and showed that readability and legibility studies also differ in the tasks and response variables they use.

The operation of distinguishing changes the conceptual resolution of a concept or theoretical account. Where a single concept previously covered several phenomena, the distinction creates separate objects, each of which can have different antecedents, consequences, measures, and explanations.

Table 4 summarizes the eight theorizing operations, and the examples presented in this section. The table specifies for each what kind of theoretical contribution can be made. Each of the theoretical operation allows for a ‘transformation’ in our thinking about a given phenomenon; it *changes* the way we look at it. This, in turn, allows us to ask new questions about a phenomenon under study, which is ultimately the trigger for new and impactful research.

3.9 Dimensions of the XYZ framework

The eight theorizing operations are organized in a circumplex. There are at least four ways in which the circumplex can be divided into subsets of related operations, each of which adds vocabulary to the operations.

3.9.1 Represent v. Predicate. A first useful way to distinguish the two sides of the circumplex is whether the theorizing operations are concerned with how a phenomenon is *represented* or what is *predicated* (see Figure 2). This distinction resembles a common separation in discussions of theory, namely between its conceptual elements (left-hand side) and the propositions or relationships formulated among them (right-hand side). Bacharach, for example, characterizes theory in terms of constructs, variables, and the relationships among them [3], while Whetten distinguishes the “what” of a theory—the concepts that constitute a topic of study—from the “how” and “why” that specify relationships and provide explanations [132].

On the representational (left-hand) side, the scholar works on the conceptual form of the phenomenon itself: giving it an identity, increasing or decreasing its level of abstraction, separating

Table 4. Summary of the eight theorizing operations

Operation	Description	Software engineering example	Theoretical contribution
Name	Establishes an observed phenomenon or pattern as a recognizable conceptual object by giving it a term through which it can be discussed and studied.	Cunningham’s <i>technical debt</i> metaphor gave a common name to the consequences of sub-optimal software design decisions to allow faster delivery [24].	Transforms an unnamed phenomenon or pattern into an object that is referable around which new questions, concepts, measurements, and explanations can accumulate.
Abstract	Recognizes several phenomena as instances of a more general concept.	The concept <i>software ecosystem</i> provides a higher-level abstraction through which work on open source communities, software product lines, software supply networks, and software business can be considered together [13, 55].	Increases conceptual generality by focusing on what is common across different observations, allowing recognition of these observations as instances of the same conceptual phenomenon.
Explain	Proposes a mechanism, process, or theoretical logic that accounts for how or why an observed phenomenon occurs.	A theory explaining why software organizations adopt different structures for development and infrastructure [66].	Deepens understanding of a phenomenon by proposing an explanation that gives rise to the phenomenon.
Connect	Establishes a relationship between concepts, theories, or bodies of knowledge that had hitherto been treated as distinct.	Control-theoretic properties with properties used in software engineering for self-adaptive systems [20].	Allows leveraging of concepts, results, and theoretical understanding developed in one body of knowledge to inform another.
Bound	Specifies the conditions, contexts, populations, or domains of application of prior established theoretical knowledge.	Effectiveness of pair programming depends on factors such as task complexity [43].	Qualifies and clarifies the scope and context in which theoretical claims apply.
Challenge	Questions an assumption, explanation, concept, or principle that has become accepted knowledge.	Parnas’s alternative criteria for decomposing systems based on information hiding, after arguing against the conventional approach based on flowcharts [82].	Enable shift in understanding to allow for new concepts and ideas that can better explain observations, and rethinking of existing concepts.
Instantiate	Takes a general theoretical account and works out what its concepts, relationships, or implications mean in a particular domain, context, or case.	Social Representations Theory in the study of code smell identification, to examine how developers collectively understand this activity [26].	Makes a theory or concept concrete in a particular setting and highlights implications and questions that may not otherwise be apparent.
Distinguish	Separates concepts or phenomena that have previously been treated as equivalent or combined within a single category.	Distinction between code <i>readability</i> and <i>legibility</i> , separating comprehension-related properties from visual properties of source code [79].	Increases conceptual resolution by revealing consequential differences, allowing the resulting concepts to have different antecedents, consequences, measures, and explanations.

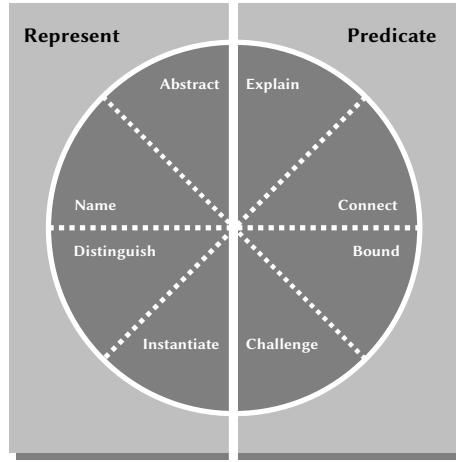


Fig. 2. Representing vs. predicating theorizing operations

concepts that have been conflated, or determining what an abstract concept or theory means in a particular setting. Swedberg, for example, treats naming and concept formation as central activities through which an observed phenomenon is turned into a theoretical object [122], while work on construct clarity emphasizes that theoretical progress depends on concepts that capture phenomena in sufficiently clear and discriminating ways [120]. Thus, Name, Abstract, Distinguish, and Instantiate operate on the representation through which the phenomenon becomes intelligible.

On the predicative (right-hand) side, theorizing works on *what can be said* about those representations: which concepts are related, how or why a phenomenon occurs, under what conditions a claim holds, and whether an existing claim or assumption must be revisited. This corresponds closely to the relational and explanatory components of theory emphasized in prior accounts of theory construction [3, 132].

Thinking of operations as representing or predicating offers a distinction in a scholar's orientation: representation is concerned with the question "what are the things we are thinking with?"; predication is concerned with the question "what can we say about them?"

3.9.2 *Constitute v. Situate.* A second division concerns whether theorizing primarily *constitutes* the phenomenon under study or *situates* it relative to something conceptual, empirical, or contextual setting (see Figure 3). This distinction resembles a difference in the literature between developing the theoretical object itself and positioning that object within a wider conceptual, empirical, or contextual setting.

Constituting operations contribute to forming the theoretical object itself: they identify a phenomenon (name), separate it from related phenomena (distinguish), place it at an appropriate level of abstraction (abstract), and develop an account of how or why it occurs (explain). Swedberg's guidelines for theorizing closely reflects this kind of work, moving from observation and naming through concept formation and typology toward explanation [122]. Conceptualization has similarly been treated as a central form of theoretical contribution because it determines how phenomena are identified, organized, and made intelligible [68].

Situating operations work on a theoretical object by placing it within a wider intellectual or empirical context. Connect links concepts or theories to others not previously linked; Bound

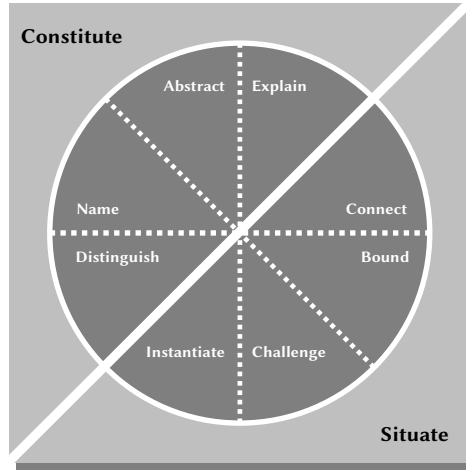


Fig. 3. Constituting vs. situating theoretical operations

specifies where its claims hold or do not hold; Challenge seeks to address anomalies in empirical observation, or to rethink common assumptions; and Instantiate works out what a concept means in a particular setting. Prior work on theory development emphasizes several forms of such relational work. Whetten treats “who, where, and when” as qualifications on theoretical claims [132], and later distinguishes between contextualizing theory and theorizing about context [133]. Alvesson and Sandberg describe problematization as identifying and questioning assumptions underlying an established theoretical account [2]; that is, problematization tends to offer new assumptions, concepts or propositions *in relation* to previous assumptions. MacInnis treats integration with other concepts and bodies of knowledge (i.e., positioning a theory or concept) as a form of conceptual contribution [68].

In sum, *constitute* develops the internal identity and logic of a theoretical object, whereas *situate* establishes how that theoretical object relates to other knowledge, competing assumptions, boundary conditions, and the contexts in which it is used.

3.9.3 Differentiate v. Unify. A third division considers whether theorizing primarily increases *differentiation* or *unifies* our understanding (see Figure 4). This distinction reflects common concerns in theory development between (a) increasing conceptual resolution by sharpening constructs and distinctions, and (b) increasing theoretical coherence by integrating concepts and relationships into a systematic (unified) account [32, 68, 120]. Prior work recognizes both kinds of contribution: theorizing may sharpen (differentiate) distinctions among constructs and phenomena, or integrate (unify) previously separate observations and concepts into a more encompassing account.

Differentiating operations increase theoretical resolution; that is, identifying a phenomenon (name) establishes it as a distinct object for inquiry. Distinguish separates concepts that might otherwise be conflated, or, it may “zoom in” on a phenomenon and identify different categories or types within [32, 68]. Instantiate specifies what a general idea means in a particular setting, thereby bringing forward specific differences from the general account. To challenge means that assumptions underpinning a theory or concept are highlighted, further raising awareness of differences with what might have been expected.

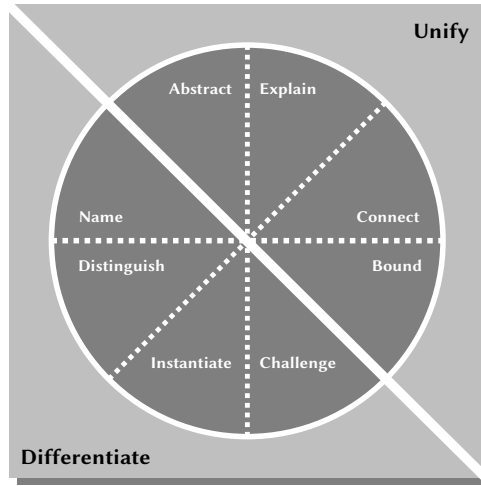


Fig. 4. Differentiating vs. unifying theorizing operations

Unifying operations, on the other hand, seek to reduce fragmentation by bringing phenomena, concepts, or claims into a more coherent structure. Abstract treats different cases as instances of a more general phenomenon; Explain organizes a set of observations through a common (thus, unified) account of how or why they occur; Connect integrates distinct concepts, theories, or bodies of knowledge; and Bound relates a claim to the conditions under which it should be expected to hold, thereby explaining why an apparently general relationship varies across settings, thus incorporating this variation into a coherent theoretical account rather than treating it as unexplained inconsistency [19]. Swedberg emphasizes abstraction and explanation as central means through which theorizing moves from disparate observations toward a more general account [122], while MacInnis identifies integration as a distinct form of conceptual contribution [68].

3.9.4 Expand v. Discipline. A fourth division concerns whether operations primarily *expand* the reach or scope of emergent theory, or that they *discipline* the emergent theory, following Weick’s notion of “disciplined imagination” [128].

Expanding operations seek to enrich, increase reach or scope of our understanding of a phenomenon. The initial operation is to identify a phenomenon (name); it expands our knowledge simply by increasing awareness of a phenomenon. Abstract extends understanding across a range of instances so as to identify a more general phenomenon. Explain expands an existing account by providing a description of *how* or *why* something occurs. Finally, connect expands the theoretical domain by leveraging knowledge of another body of knowledge. Whetten treats comprehensiveness (“*are all relevant factors included?*” [132, p. 490]) as one of the recurring concerns in theory development. Bacharach points out that “*constructs and variables with broader scope allow hypotheses and propositions to have greater overall explanatory power*” [3, p.507]. Likewise, MacInnis identifies abstraction and integration as forms of conceptual contribution because they extend understanding beyond singular observations or concepts [68]. Thus, theorizing can progress by enlarging the set of phenomena, relationships, and bodies of knowledge that a theoretical account encompasses.

Disciplining operations constrain, qualify, sharpen, or ‘narrow’ (rather than expand) theoretical claims or scope. Bound identifies boundary conditions, which constrains the domain or context

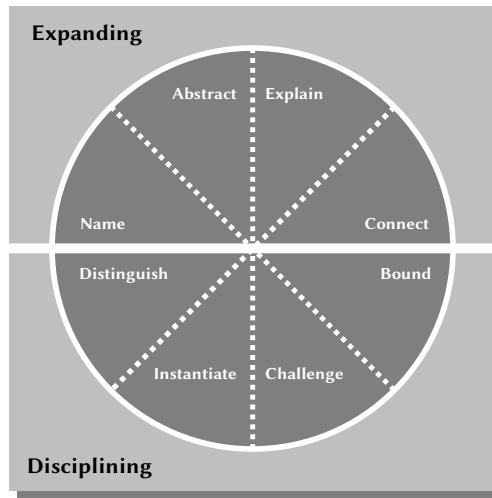


Fig. 5. Expanding vs. disciplining theorizing operations

where a theory or concept is applicable [19]. Challenge problematizes assumptions or explanations in order to question foundational beliefs [2]. Instantiate ‘confronts’ a general account with the particulars of a concrete setting; and Distinguish prevents overly broad concepts from collapsing meaningfully different phenomena into one category.

3.10 The X, Y, and Z of Theorizing in Software Engineering

Theorizing serves a number of purposes. The circumplex in Fig. 1 highlights three desiderata of theorizing, which are desirable qualities to consider during theorizing. Theory is expected to reach beyond a particular case (extension, denoted ‘X’), to organize otherwise fragmented knowledge (synthesis, denoted ‘Y’), and to remain sufficiently ‘responsive’ to the particulars of the empirical contexts in which it is applied (realization, denoted ‘Z’).

These three desiderata should not be read as a complete list of qualities of good theory. Common criteria such as parsimony, clarity, coherence, and falsifiability, apply to theoretical accounts more generally. Parsimony, for example, is concerned with the ‘economy’ of a theoretical account, i.e., the question “*should some factors be deleted because they add little additional value to our understanding?*” [132, p. 490]. Whetten writes that “*sensitivity to the competing virtues of parsimony and comprehensiveness is the hallmark of a good theorist*” [132, p. 490]. Parsimony is a cross-cutting criterion for evaluating theory and theoretical products rather than a distinct goal of theorizing.

Extension (X). Concepts (and theories) are valued partly for their scope: they allow understanding developed in one setting to extend beyond that setting. Discussions of theory quality have therefore emphasized generality, scope, and the balance between comprehensiveness and parsimony [3, 132]. Sartori describes high level categories as having “maximal extension” [98, p. 1044]. Abstract has the greatest potential for extension because it deliberately removes case-specific detail in order to identify what different cases have in common. Extension is concerned with the question: *What is the scope of a concept or theory?*

Synthesis (Y). Theory is expected to organize knowledge rather than leave findings as isolated observations. Integrative conceptual contributions connect previously separate concepts or bodies of knowledge and make the underlying relationships within a domain more visible [68]. Jaakkola

describes theory synthesis as achieving “conceptual integration” [51, p. 22] across theories or literature streams, bringing disparate elements into “a more coherent whole” [51, p. 21]. More generally, integrative theoretical frameworks allow findings from otherwise separate research strands to contribute to a cumulative tradition. Connect has the greatest potential for synthesis because it explicitly relates concepts, explanations, findings, or theories that would otherwise remain separate. Synthesis is concerned with the question: *How much knowledge can be brought into a single coherent theoretical account?*

Realization (Z). Realization concerns the extent to which theoretical understanding remains adequate to the particulars of the empirical settings in which a phenomenon occurs. Applying an existing theory in a new domain can reveal assumptions, relationships, mechanisms, or contextual conditions that were not apparent or possibly relevant in the setting in which the theory was originally developed [133, 138]. The aim of cross-context theorizing is not only to apply theory, but rather to identify new questions through which the theory itself can be refined or extended [133]. Instantiate has the greatest potential for this form of realization because it works out what a general theoretical account means in a particular empirical context. In doing so, it can reveal discrepancies, missing elements, and context-specific conditions that become objects of further theorizing. Realization is concerned with the question: *What becomes visible when a general theoretical account is made concrete here?*

Table 5 summarizes the focus of extension (X), synthesis (Y), and realization (Z), which also emphasizes the differences between these three desiderata. The table indicates the operation with maximum potential for that desideratum.

Table 5. The X, Y, and Z of theorizing: extension, synthesis, and realization

Desideratum	Maximum	Direction	Transformation
Extension (X)	Abstract	From particular to general	Moves from specific observations, cases, or concepts toward a more general theoretical category or account by identifying what they have in common.
Synthesis (Y)	Connect	From theory to theory	Establishes a theoretically meaningful relationship between distinct theoretical elements that remain conceptually separate, such as constructs, concepts, explanations, theories, or bodies of knowledge.
Realization (Z)	Instantiate	From general to particular	Works out how a more general theoretical concept, relationship, or account is realized in a particular empirical setting.

4 Example: Theorizing in Software Testing

The operations introduced above need not occur in a fixed sequence, nor does a single study necessarily perform only one operation. To illustrate this, we trace the development of a small subset of the literature on mock objects. The literature begins from a practical problem in unit testing: a unit under test often depends on other components whose behavior is difficult, expensive, or undesirable to reproduce during a test. As the discussion below demonstrates, the study of this problem led to research that used all eight theorizing operations of the XYZ framework, and through these studies each operation gave rise to new research questions.

Table 6 summarizes the examples. We note that the order of presentation differs from that of the framework above, so as to reconstruct the theoretical contributions in the most suitable order. The examples are an analytical reconstruction: the cited authors do not use our terminology for the theorizing operations. Furthermore, we illustrate each operation with only one study, and so this reconstruction is not meant to give a complete account of the research on mock objects.

4.1 Name

Mackinnon et al. [69] proposed mock objects as a technique that replaces “domain code with dummy implementations that emulate real code.” Mock objects allow developers to isolate software from difficult external dependencies making unit-testing easier [123]. By coining the term and presenting the idea, mock objects became a recognizable object of inquiry in software testing. The rapid emergence of a literature around mock objects indicates how naming a new phenomenon can have a generative effect; researchers start to ask questions about the new phenomenon. Within the next few years, researchers documented recurring patterns for their use [17], investigated whether they could be generated automatically for test factoring [95], and revisited their role in software design [35].

4.2 Distinguish

Pereira and Hora [83] distinguished manually implemented mock classes from mocks dynamically created by mocking frameworks. Although both are used to replace objects during testing, they differ in how they are created, structured, reused, and maintained. Treating manually implemented mock classes as a distinct phenomenon makes those differences available for empirical investigation.

Making these differences explicit opened up comparative questions about the different approaches. Mostafa and Wang [75], for example, investigated how different mocking frameworks are used in practice. Xiao et al. [137] further investigated when developers use mocking frameworks and when they rely on alternative approaches. The original distinction drawn by Pereira and Hora thus led to a series of new questions that were subsequently investigated.

4.3 Challenge

Solms and Marshall [106] questioned an assumption implicit in conventional mocking: that a mock can adequately replace a real component when its behavior is specified within the test. They pointed out that existing mocking frameworks offer no guarantee that the behavior encoded in a mock is consistent with the behavior required of the component it replaces. They proposed contract-based mocking as a way of constraining mocks by the service contracts of the real components.

Questioning the adequacy of unconstrained mocks opened up research on whether mocked behavior remains valid as software changes. Subsequent studies investigated how automatically generated stubs can become incomplete or obsolete and how they can be repaired [140], how mock assertions relate to the behavior that tests are actually able to verify [139], and how limitations of existing mocking frameworks led developers to adopt alternative mocking practices [137].

4.4 Explain

Experience with mock objects suggested another effect of their use; Freeman et al. [35] observed that mock-based development could improve the structure of code by clarifying interactions, reducing dependencies, and supporting the discovery of interfaces. Freeman et al. proposed that the effect arises because testing an object before its collaborators have been implemented requires developers to specify the services those collaborators must provide and the interactions that should occur. These expectations make collaborator roles explicit, which can then be represented through interfaces and implemented by subsequent objects. Thus, rather than simply observing a link between use of mock objects and improved code structure, Freeman et al. offered an explanation of why this is so, thus making a theoretical contribution to the literature on mock objects.

Once this mechanism had been proposed, its constituent claims became available for further investigation. Nandigam et al. [76], for example, examined whether mock objects can support interface-based object-oriented design, following up on Freeman et al.’s claim that interaction

requirements can help reveal interfaces. Choma et al. [22] subsequently investigated developers' experiences of using test-driven development (TDD) and mock objects for design, including whether the practice helped them identify responsibilities, interactions, and interfaces during development.

4.5 Abstract

Meszaros [72] identified several related concepts, namely dummy object, stub, and mock object, and abstracted these as instances of what he called *test doubles*. Their differences remain, but they share a common underlying structure: each replaces a component of a production system for the purposes of testing. This abstraction created a new level at which the phenomenon can be studied. Rather than asking only about mocks, stubs, or dummies separately, researchers could now investigate test doubles as a category. Fazzini et al. [30], for example, investigated how test doubles are created and used in Android applications, which types occur in practice, and how their use differs from conventional Java systems. More recent work has treated problems arising from test doubles themselves as research problems, including how mismatches between expected and actual invocations can be detected and resolved [67].

4.6 Instantiate

Fazzini et al. [30] took the concept of a test double and examined how it is realized in the specific setting of Android testing. Their study shows that the Android context matters: developers use test doubles to replace platform and environment-facing classes, and the mix and use of different forms of test double differs in some respects from that observed in conventional (non-Android) Java systems. The study therefore does more than demonstrating that test doubles are also used on Android; it shows how the general concept takes on specific forms in response to characteristics of Android as a platform.

Making the concept concrete also revealed problems that are less visible at the abstract level. Fazzini et al. identified cases in which test doubles introduced test smells and mistakes in test code, and argued that obsolete test doubles and code-test co-evolution warranted further investigation [30]. These findings motivated further questions about how obsolete or mismatched test doubles can be detected, how they should evolve with the code they replace, and which platform characteristics create particular risks. More recent work has begun to address these questions by investigating mismatched invocations on test doubles and techniques for resolving them automatically [67].

4.7 Connect

Galler et al. [37] connected mock objects with the concept of design-by-contract [73]. Design-by-contract specifies the obligations and guarantees of software components through pre-conditions, post-conditions, and invariants. Galler et al. used this idea to address a problem in mocking: the behavior of a mock is normally specified separately from the behavior of the component it replaces. By deriving mock behavior from the component's contract, they use an established theory of behavioral specification to constrain and generate mock objects.

The connection changed how mocking can be understood. A mock is no longer only a manually programmed stand-in for another component; rather, its behavior can be related systematically to an independent specification of what that component is allowed to do. This opened up research on specification-driven mocking. Samimi et al. [96], for example, explicitly built on earlier specification-based approaches and investigate whether mock behavior can instead be expressed declaratively and executed using constraint solving. By connecting different bodies of literature, knowledge of one field can be leveraged in another.

4.8 Bound

Spadini et al. [107] investigated when developers choose to use mock objects and when they prefer to use real dependencies. Their findings showed that the usefulness of mocking is conditional. Developers commonly mock dependencies that are difficult to configure or that access external resources, such as databases and web services, but are less inclined to mock objects that they can easily control. Most importantly, when the purpose of the test is to exercise the integration with a dependency itself, developers generally avoid mocking that dependency and use the real implementation instead. The study therefore qualified the general usefulness of mocking by identifying conditions under which substitution is, and is not, seen as useful.

Specifying these conditions changed the research question from *whether* mocking is useful to *when* it is useful. This opened up further research on the circumstances that shape mocking decisions. Xiao et al. [137], for example, explicitly built on Spadini et al. and investigated mocking across Apache projects, including when developers use mocking frameworks, when they resort to customized mock classes, and what limitations influence these choices.

Table 6. Example use of theorizing operations in the domain of software testing

Operation	Example	Theoretical contribution	Subsequent research
Name	Mackinnon et al. [69]	Established mock objects as a recognizable testing concept for replacing objects or components during testing.	Subsequent work examined how mock objects could isolate difficult external dependencies [123], identified recurring patterns in their use [17], and investigated whether they could be generated automatically [95].
Distinguish	Pereira and Hora [83]	Distinguished manually implemented mock classes from mocks generated dynamically by mocking frameworks, treating them as different phenomena with different characteristics and uses.	The distinction made it possible to compare how alternative forms of mocking were designed, reused, and maintained. Later work examined when developers used mocking frameworks and when they resorted to alternative forms of mocking [137].
Challenge	Solms and Marshall [106]	Questioned the assumption that mock behavior specified independently of the real component necessarily provided an adequate substitute for that component.	The challenge made the validity and maintenance of mocked behavior an explicit research problem. Later work examined incomplete or obsolete stub behavior [140] and the behavior that mock assertions were actually able to verify [139].
Explain	Freeman et al. [35]	Explained how mock-based testing could influence object-oriented design: specifying expected interactions exposed collaborator roles, which could then be represented as interfaces and shape the emerging design.	The proposed mechanism made its intermediate claims researchable. Subsequent work investigated whether mock objects supported interface-based object-oriented design [76] and how developers experienced mock objects as part of test-driven design [22].
Abstract	Meszaros [72]	Recognized dummy objects, test stubs, and mock objects as instances of the more general concept of a test double: a test-specific stand-in for a production component.	The abstraction created a level at which test doubles could be studied as a class rather than only as separate techniques. Subsequent work investigated variation among test doubles in Android systems [30] and problems that arose across test doubles, such as mismatched invocations [67].

Continued on next page

Table 6 continued from previous page

Operation	Example	Theoretical contribution	Subsequent research
Instantiate	Fazzini et al. [30]	Took the general concept of a test double and examined how it was realized in the specific context of Android testing, revealing platform-specific forms, usage patterns, and failure modes.	Making the concept concrete revealed problems such as obsolete or problematic test doubles, test smells, and mismatches between stubbed and exercised behavior. Later work investigated how mismatched invocations could be detected and resolved automatically [67].
Connect	Galler et al. [37]	Connected mock objects with the distinct body of knowledge of Design by Contract [73], using contract specifications to constrain and generate mock behavior.	The connection made concepts from behavioral specification available for reasoning about mocking. Samimi et al. [96], for example, investigated whether mock behavior could be expressed declaratively and executed using constraint solving rather than programmed manually.
Bound	Spadini et al. [107]	Identified conditions under which mocking was more or less appropriate.	This shifted inquiry from whether mocking was useful to when it was useful. Xiao et al. [137] extended this question across Apache projects, examining when developers used mocking frameworks, when they used customized mocks, and what limitations influenced these choices.

5 Discussion and Conclusion

In this article we address the apparent lack of theory in the software engineering discipline. Several scholars within the community have lamented that SE lacks theory; despite several initiatives and proposals, the field does not have a focus on theory. Research studies frequently study new technologies and topics without attempting to consider how those might be instances of higher-level concepts that had previously been studied. The field's approach to building a cumulative tradition is mostly the citation of related work without synthesizing this work. The methodological traditions of SE tend to ignore the role of theory, primarily because it is not the norm.

This article offers a different and optimistic perspective by making two arguments. First, instead of focusing on theory as a product, we argue that for SE to mature as a discipline, there is great potential in the focus on the theorizing process. Second, the SE literature lacks an awareness of the theorizing it does, partly due to a lack of norms on how research is done in our field and how we train new researchers, but also due to a lack of vocabulary. This article presents a novel framework comprising and positioning eight theorizing operations which can be considered general theorizing activities also found in other disciplines, and demonstrates that these operations are also performed by SE researchers. That is to say, the theorizing activities that occur in other, more mature fields can also be found within SE research. The framework therefore can help to make the theorizing explicit and visible, and to increase researchers' awareness, in a similar way to how the empirical research tradition found widespread adoption within the community in the early 2000s. An increased awareness, in turn, can help researchers to theorize more deliberately, which we argue, will help increase the field's focus on creating longer-lasting knowledge.

5.1 Implications

5.1.1 Revisiting Study Motivation. A common approach to motivate research studies in SE is what Alvesson and Sandberg have labeled 'gap-spotting' [2]. That is, researchers observe that "nobody studied X in the context of Y, and so we fill this gap by conducting study Z." While some gaps

in knowledge require attention, other gaps are better left open: not everything is worthy of our attention.

A theory-oriented perspective suggests an alternative starting point: rather than investigating what has not been studied, studies should establish what to examine next, given our current knowledge on a topic. Each of the eight operations presented in this article provides a different way of answering that question. A newly observed phenomenon may need to be *named* or *distinguished* from something already known. An empirical regularity may require *explanation*. A field of inquiry may be leveraged by using insights from another field, or simply by establishing relationships (connect), or established relationships may need to be qualified (bound), instantiated in a new setting, or challenged by empirical observations that do not fit current explanations.

The eight theorizing operations help us to take smaller but practical steps than building complete theories. Theorizing helps us to identify the next research question or to design the next study in a way that can help build a cumulative tradition. Doing so is particularly important for our field, given the rapid technological advances and the ever-increasing number of studies. While new technologies legitimately give rise to new questions, technological novelty does not imply theoretical novelty. The SE field has a tendency to study new technologies without consideration of prior knowledge. Rather than starting with a clean slate, a more useful approach would be to ask of what general phenomenon the new technology, topics, or events are an instance. This change in orientation allows for a cumulative tradition.

5.1.2 Revisiting Study Contributions. A second implication concerns how contributions are described. Software engineering papers commonly report contributions such as techniques, tools, datasets, taxonomies, empirical findings, or frameworks. Empirical findings, in particular, are frequently tied to the specific context in which they are established. While these can be valuable contributions, we observe a potential missed opportunity to articulate more clearly in which way those findings contribute to prior knowledge in a theoretical sense. For example, do those findings *bound* or *challenge* prior insights? Might they add an *explanation*?

We contend that recognizing the theorizing operation performed by a study can make its contribution to theory more visible. The framework encourages researchers to consider which of the eight theorizing operations may be applicable; in other words, an awareness of typical theorizing activities across a variety of scientific disciplines can also help software engineering researchers to reflect on their work and address the question: *What kind of theoretical transformation does the study represent?*

Theorizing frequently occurs over a series of papers, or a whole body of work. For example, a new phenomenon may be *named* and identified as a concept in one paper, connected to a different literature in another, and distinguished from other concepts in another paper still. In this manner, theories may emerge over a series of studies, conducted by different scholars, rather than developed and presented within a single study by a single research team. The example of theorizing in software testing presented in Sec. 4 has illustrated how different authors, who did not claim to do any theory development, together developed an increasingly rich theoretical understanding of mock objects and test doubles.

The XYZ framework offers a vocabulary for reconstructing this process. Instead of asking only whether one study replicates or extends another, researchers can ask what theoretical transformation occurred between them. Did a later study generalize the phenomenon, explain it, qualify its scope, challenge an assumption, or distinguish cases that the earlier account treated together? Each of these operations contributes deeper insights, allowing a cumulative body of knowledge to grow, rather than listing a chronological sequence of studies.

Research programmes could consequently be organized more explicitly around emergent theoretical concepts and unexplored theoretical activities. A mature programme should not focus only on producing additional findings in an incremental fashion, but rather it should progressively change what is understood about that phenomenon through explanation, connection, differentiation, qualification, and challenge. The eight operations provide one vocabulary for making such transformations visible.

5.1.3 Revisiting Literature Reviews. Like others before us, we observe that many systematic reviews and mapping studies lack depth and synthesis. While annotated bibliographies and survey papers that provide trends can offer value, the promise of systematic literature reviews as originally envisaged has, in our opinion, not delivered. Recognizing theoretical contributions by identifying the theorizing operations that studies have performed can help in the synthesis process of a systematic review. It provides the “terminological glue” that can help deliver a cumulative science of software engineering.

A review can reconstruct the theoretical conversation [112] taking place across studies. It can identify which concepts have been introduced, which explanations have been proposed, which claims have acquired boundary conditions, where apparently conflicting findings can be reconciled or resolved, and which assumptions have been challenged. In this role, reviewing can go beyond the current practice of describing prior literature.

The eight operations provide one possible lens for such reviews. A review might reveal, for example, that a literature contains extensive instantiation of a concept across settings but little attempt to explain the phenomenon; or that many studies repeatedly identify associations without connecting them into a broader account. Similarly, identification of boundary conditions can help explain the apparent inconsistencies.

We do not suggest that every systematic review should generate new theory, but rather that the level of depth of both the questions asked and the answers delivered is increased. Instead of asking “what has been studied?” we could more specifically focus on boundary conditions, clarification and distinctions of concepts, or challenge prior findings.

5.1.4 Revisiting Research Evaluation. Recognizing theorizing also has implications for reviewing and evaluating research. Current evaluation criteria include a heavy emphasis on novelty and other aspects that are readily visible: a new technique, a new dataset, or a statistically significant effect. Theoretical contributions such as implied by using the theorizing operations presented in this paper can be more difficult to recognize. The vocabulary offered in this article supports both authors and reviewers to articulate and recognize what a study has contributed. We caution that the XYZ framework should not be interpreted as a checklist; it may well be that studies do not use any of the theorizing operations presented in this paper.

5.1.5 Revisiting Software Engineering Research Training. There is also an implication for how software engineering researchers are trained. Doctoral training includes considerable attention to research design, measurement, sampling, statistical analysis, qualitative analysis, and threats to validity. However, there appears to be less attention for the role of theory in SE, partly because there are few guidelines that have found widespread adoption, suggesting the community has not recognized those to be fitting. As a result, SE researchers may therefore become highly skilled at answering a research question without receiving equally systematic guidance on how to make theoretical contributions.

We contend that theorizing can be taught as part of software engineering research training. Researchers can learn to ask what phenomenon they are studying, to identify or distinguish concepts, to propose explanations where these are missing, to ask what other knowledge should be

connected, and what evidence might challenge prior findings. To a large extent, SE researchers already do this. However, the XYZ framework provides a vocabulary for making this discussion more explicit. We also envisage that this would change the way doctoral students engage with prior work. Rather than listing all prior work in terms of their substantive contributions, positioning prior work as theoretical contributions to an emergent theory may be more productive so as to better identify and justify the intellectual gap that requires further research.

5.2 Limitations of the XYZ Framework

It is worth pointing out the limitations (or ‘bound’) the assertions made about the XYZ framework in a number of ways. First, we do not assert that the eight operations defined by the XYZ framework represent a complete spectrum of all theoretical operations that might be available to the software engineering scholar. However, based on the synthesis underpinning the framework, no categorically different operations were identified. Second, not all studies seek to make theoretical contributions, and so not all studies can necessarily be classified according to the XYZ framework. Other types of contributions, such as measurement, infrastructure, datasets, and replications also offer valuable contributions, albeit not theoretical. Third, the vocabulary offered by the XYZ framework is a single representation, and a synthesis by one researcher; as such, it is not a definitive account of what theorizing operations might entail. Rather, it is offered as a practical framework that can assist other researchers to develop their own sensitivity to theorizing. Fourth, the circumplex should not be interpreted as a fixed sequence through which theory must develop. Operations may be combined in any order; a single study may adopt one or more operations. Finally, while the framework seeks to increase awareness and recognition, those alone do not ensure a cumulative science. Making a theoretical contribution explicit does not guarantee that later researchers will use it. Rather, we argue that recognition enables others: theoretical work that is visible can be extended, evaluated, and challenged more deliberately than theoretical work that remains implicit.

5.3 The Future of Software Engineering Research

The software engineering discipline is, at the time of writing, undergoing a major transformation. Apart from the discipline-specific changes that generative AI technologies are introducing to the nature of software engineering as a professional practice, the SE research community is itself facing a challenge as the number of research studies being published is growing at unprecedented speed. With so many new empirical observations being reported, one must wonder: how does it all fit together? Will the SE community be able to recognize its theoretical forest for its fast growing number of empirical trees?

This article seeks to offer an optimistic perspective by pointing out that the community does, in fact, make theoretical advances, but that these are not currently widely recognized as such. The SE community lacks a suitable vocabulary that fits its research traditions, and yet, when looking more closely, software engineering research as a scientific discipline is not all that different from other fields that do have stronger cumulative traditions. Drawing on guidance from a wide range of other scientific discourses, this article offers a vocabulary for a set of eight theoretical operations, which represent ‘transformations’ that can contribute to a cumulative body of knowledge. The eight operations are organized in a circumplex according to a number of internally consistent dimensions.

It is our vision that a better recognition of the theoretical contributions made by the many excellent studies of the SE field can help make the field’s theoretical foundations more visible. Similar to the rise of empirical methods and the sophistication of research techniques within our field over the past few decades, an increased awareness of theorizing operations may help evolve the field towards what one could call a ‘theory-oriented software engineering,’ following the success

of evidence-based software engineering [62, 115]. To be clear, not every SE research paper needs to theorize; but it is our assertion that studies that do align with a theorizing operation may be able to make a longer-lasting contribution that helps to build a cumulative tradition.

Declaration on the use of Generative AI

ChatGPT was used as a search tool to identify potential relevant literature, both for the development of the XYZ framework and for identifying relevant examples. All identified and cited sources were read and analyzed by the author. All text of this manuscript was manually written and edited by the author. ChatGPT was further used to check references and identify grammar and spelling mistakes.

Acknowledgments

This manuscript is part of a larger research programme that seeks to clarify the role of theory in software engineering. The author thanks his many colleagues and collaborators over the years, including those at the GTSE workshop series, as well as organizers of various events that offered an arena to deliberate on this, most recently a Research Seminar at the Federal University of Pernambuco (UFPE) in Recife, Brazil, in 2025, and CBSOFT 2026 in São Paulo, Brazil. For the purpose of Open Access, the author has applied a CC BY public copyright license to any Author Accepted Manuscript version arising from this submission.

References

- [1] Faheem Ahmed and Luiz Fernando Capretz. 2007. Institutionalization of Software Product Line: An Empirical Investigation of Key Organizational Factors. *Journal of Systems and Software* 80, 6 (2007), 836–849. doi:10.1016/j.jss.2006.09.010
- [2] Mats Alvesson and Jörgen Sandberg. 2011. Generating Research Questions Through Problematisation. *Academy of Management Review* 36, 2 (2011), 247–271. doi:10.5465/amr.2009.0188
- [3] Samuel B. Bacharach. 1989. Organizational Theories: Some Criteria for Evaluation. *Academy of Management Review* 14, 4 (1989), 496–515. doi:10.5465/amr.1989.4308374
- [4] Astri Barbala, Tor Sporse, and Klaas-Jan Stol. 2024. A case study of continuous adoption in the Norwegian public sector. In *Proceedings of the 57th Hawaii International Conference on System Sciences*. doi:10.24251/HICSS.2024.248
- [5] Jeremy Hugh Baron. 2009. Sailors’ scurvy before and after James Lind—a reassessment. *Nutrition reviews* 67, 6 (2009), 315–332. doi:10.1111/j.1753-4887.2009.00205.x
- [6] Michel Beaudouin-Lafon, Susanne Bødker, and Wendy E. Mackay. 2021. Generative Theories of Interaction. *ACM Transactions on Computer-Human Interaction* 28, 6, Article 45 (2021), 54 pages. doi:10.1145/3468505
- [7] Joseph Black. 1803. *Lectures on the Elements of Chemistry*. Longman and Rees, Edinburgh. 2 volumes.
- [8] George Boole. 1854. *An Investigation of the Laws of Thought, on Which Are Founded the Mathematical Theories of Logic and Probabilities*. Walton and Maberly, London.
- [9] Paulo Borba, Leopoldo Teixeira, and Rohit Gheyi. 2012. A theory of software product line refinement. *Theoretical Computer Science* 455 (Oct. 2012), 2–30. doi:10.1016/j.tcs.2012.01.031
- [10] Denny Borsboom, Han L. J. van der Maas, Jonas Dalege, Rogier A. Kievit, and Brian D. Haig. 2021. Theory Construction Methodology: A Practical Framework for Building Theories in Psychology. *Perspectives on Psychological Science* 16, 4 (2021), 756–766. doi:10.1177/1745691620969647
- [11] Jan Bosch. 2000. Organizing for Software Product Lines. In *Software Architectures for Product Families (Lecture Notes in Computer Science, Vol. 1951)*, Frank van der Linden (Ed.). Springer, 117–134. doi:10.1007/978-3-540-44542-5_15
- [12] Jan Bosch. 2001. Software Product Lines: Organizational Alternatives. In *Proceedings of the 23rd International Conference on Software Engineering*. 91–100. doi:10.1109/ICSE.2001.919084
- [13] Jan Bosch. 2009. From Software Product Lines to Software Ecosystems. In *Proceedings of the 13th International Software Product Line Conference*. 111–119.
- [14] Jan Bosch, Gert Florijn, Danny Greefhorst, Juha Kuusela, J. Henk Obbink, and Klaus Pohl. 2002. *Variability Issues in Software Product Lines*. Springer Berlin Heidelberg, 13–21. doi:10.1007/3-540-47833-7_3
- [15] Louis J Bourgeois III. 1979. Toward a method of middle-range theorizing. *Academy of Management Review* 4, 3 (1979), 443–447. doi:10.5465/amr.1979.4289127
- [16] Dermot Breslin and Caroline Gatrell. 2023. Theorizing Through Literature Reviews: The Miner–Prospector Continuum. *Organizational Research Methods* 26, 1 (2023), 139–167. doi:10.1177/1094428120943288

- [17] Matthew A. Brown and Eli Tapolcsanyi. 2003. Mock Object Patterns. In *Proceedings of the 10th Conference on Pattern Languages of Programs (PLoP 2003)*. The Hillside Group.
- [18] Andrew Burton-Jones, Ephraim R. McLean, and Emmanuel Monod. 2015. Theoretical Perspectives in IS Research: From Variance and Process to Conceptual Latitude and Conceptual Fit. *European Journal of Information Systems* 24, 6 (2015), 664–679. doi:10.1057/ejis.2014.31
- [19] Christian Busse, Andrew P. Kach, and Stephan M. Wagner. 2017. Boundary Conditions: What They Are, How to Explore Them, Why We Need Them, and When to Consider Them. *Organizational Research Methods* 20, 4 (2017), 574–609. doi:10.1177/1094428116641191
- [20] Ricardo Diniz Caldas, Arthur Rodrigues, Eric Bernd Gil, Genáina Nunes Rodrigues, Thomas Vogel, and Patrizio Pelliccione. 2020. A hybrid approach combining control theory and AI for engineering self-adaptive systems. In *Proceedings of the IEEE/ACM 15th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS '20)*. ACM, 9–19. doi:10.1145/3387939.3391595
- [21] Shyam R Chidamber and Chris F Kemerer. 1994. A metrics suite for object oriented design. *IEEE Transactions on software engineering* 20, 6 (1994), 476–493. doi:10.1109/32.295895
- [22] Joelma Choma, Eduardo M. Guerra, and Tiago Silva da Silva. 2018. Developers' Initial Perceptions on TDD Practice: A Thematic Analysis with Distinct Domains and Languages. In *Agile Processes in Software Engineering and Extreme Programming (Lecture Notes in Business Information Processing, Vol. 314)*, Juan Garbajosa, Xiaofeng Wang, and Ademar Aguiar (Eds.). Springer, Cham, 68–85. doi:10.1007/978-3-319-91602-6_5
- [23] Paul Clements and Linda Northrop. 2001. *Software Product Lines: Practices and Patterns*. Addison-Wesley, Boston, MA.
- [24] Ward Cunningham. 1992. The WyCash Portfolio Management System. *ACM SIGPLAN OOPS Messenger* 4, 2 (1992), 29–30. OOPSLA'92. Addendum to the Proceedings. doi:10.1145/157710.157715
- [25] Michael A. Cusumano. 2004. *The Business of Software: What Every Manager, Programmer, and Entrepreneur Must Know to Thrive and Survive in Good Times and Bad*. Free Press, New York.
- [26] Rafael Maiani de Mello, Anderson G. Uchôa, Roberto Felício Oliveira, William Nalepa Oizumi, Jairo Souza, Kleyson Mendes, Daniel Oliveira, Balduino Fonseca, and Alessandro Garcia. 2019. Do Research and Practice of Code Smell Identification Walk Together? A Social Representations Analysis. In *Proceedings of the 13th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*. 1–6. doi:10.1109/ESEM.2019.8870141
- [27] Robert Dubin. 1978. *Theory Building* (revised ed.). Free Press, New York.
- [28] Kathleen M Eisenhardt. 1989. Building theories from case study research. *The Academy of Management Review* 14, 4 (1989), 532–550. doi:10.5465/amr.1989.4308385
- [29] Barbara Farbey and Anthony Finkelstein. 1999. Exploiting Software Supply Chain Business Architecture: A Research Agenda. In *Proceedings of the 1st Workshop on Economics-Driven Software Engineering Research*. Los Angeles, CA.
- [30] Mattia Fazzini, Chase Choi, Juan Manuel Copia, Gabriel Lee, Yoshiki Kakehi, Alessandra Gorla, and Alessandro Orso. 2022. Use of Test Doubles in Android Testing: An In-Depth Investigation. In *Proceedings of the 44th International Conference on Software Engineering (ICSE '22)*. Association for Computing Machinery, 2266–2278. doi:10.1145/3510003.3510175
- [31] Michael Felderer and Guilherme Horta Travassos (Eds.). 2020. *Contemporary empirical methods in software engineering*. Springer Cham. doi:10.1007/978-3-030-32489-6
- [32] Greg Fisher and Herman Aguinis. 2017. Using Theory Elaboration to Make Theoretical Advancements. *Organizational Research Methods* 20, 3 (2017), 438–464. doi:10.1177/1094428116689707
- [33] Brian Fitzgerald and Klaas-Jan Stol. 2017. Continuous software engineering: A roadmap and agenda. *Journal of Systems and Software* 123 (2017), 176–189. doi:10.1016/j.jss.2015.06.063
- [34] Bernard K Forscher. 1963. Chaos in the brickyard. *Science* 142, 3590 (1963), 339–339.
- [35] Steve Freeman, Tim Mackinnon, Nat Pryce, and Joe Walnes. 2004. Mock Roles, Not Objects. In *Companion to the 19th Annual ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA '04)*. Association for Computing Machinery, New York, NY, USA, 236–246. doi:10.1145/1028664.1028765
- [36] Hans Fuchs, Michele D'Anna, and Federico Corni. 2022. Entropy and the Experience of Heat. *Entropy* 24, 5 (May 2022), 646. doi:10.3390/e24050646
- [37] Stefan J. Galler, Andreas Maller, and Franz Wotawa. 2010. Automatically Extracting Mock Object Behavior from Design by Contract Specification for Test Data Generation. In *Proceedings of the 5th International Workshop on Automation of Software Test*. Association for Computing Machinery, New York, NY, USA, 43–50. doi:10.1145/1808266.1808273
- [38] Archibald E. Garrod. 1902. The Incidence of Alkaptonuria: A Study in Chemical Individuality. *The Lancet* 160, 4137 (1902), 1616–1620.
- [39] Barney G. Glaser. 2002. Conceptualization: On Theory and Theorizing Using Grounded Theory. *International Journal of Qualitative Methods* 1, 2 (2002), 23–38. doi:10.1177/160940690200100203

- [40] Barney G. Glaser and Anselm L. Strauss. 1967. *The Discovery of Grounded Theory: Strategies for Qualitative Research*. Aldine Publishing Company, Chicago, IL.
- [41] Robert L. Glass. 2002. *Facts and Fallacies of Software Engineering*. Addison-Wesley Professional.
- [42] Shirley Gregor and David Jones. 2007. The Anatomy of a Design Theory. *Journal of the Association for Information Systems* 8, 5 (2007), 312–335. doi:10.17705/1jais.00129
- [43] Jo E. Hannay, Tore Dybå, Erik Arisholm, and Dag I. K. Sjøberg. 2009. The Effectiveness of Pair Programming: A Meta-Analysis. *Information and Software Technology* 51, 7 (2009), 1110–1122. doi:10.1016/j.infsof.2009.02.001
- [44] Nik Rushdi Hassan, Paul Benjamin Lowry, and Lars Mathiassen. 2022. Useful Products in Information Systems Theorizing: A Discursive Formation Perspective. *Journal of the Association for Information Systems* 23, 2 (2022), 418–446. doi:10.17705/1jais.00730
- [45] Carl G Hempel. 1952. *Fundamentals of Concept Formation in Empirical Science*. University of Chicago Press.
- [46] Rashina Hoda. 2022. Socio-technical grounded theory for software engineering. *IEEE Transactions on Software Engineering* 48, 10 (2022), 3808–3832. doi:10.1109/TSE.2021.3106280
- [47] Giora Hon and Bernard R. Goldstein. 2013. J. J. Thomson’s plum-pudding atomic model: The making of a scientific myth. *Annalen der Physik* 525, 8–9 (2013), A129–A133. doi:10.1002/andp.201300732
- [48] Giora Hon and Bernard R. Goldstein. 2024. The Roles of Thomson and Rutherford in the Birth of Atomic Physics: The Interaction of Experiment and Theory. *Annalen der Physik* 536, 8, Article 2400090 (2024). doi:10.1002/andp.202400090
- [49] Kristina Höök and Jonas Löwgren. 2012. Strong Concepts: Intermediate-Level Knowledge in Interaction Design Research. *ACM Transactions on Computer-Human Interaction* 19, 3, Article 23 (2012), 18 pages. doi:10.1145/2362364.2362371
- [50] Martin Höst and Alma Oručević-Alagić. 2011. A systematic review of research on open source software in commercial software product development. *Information and Software Technology* 53, 6 (2011), 616–624. doi:10.1016/j.infsof.2010.12.009
- [51] Elina Jaakkola. 2020. Designing Conceptual Articles: Four Approaches. *AMS Review* 10, 1–2 (2020), 18–26. doi:10.1007/s13162-020-00161-0
- [52] Victoria Jackson, Bogdan Vasilescu, Daniel Russo, Paul Ralph, Rafael Prikladnicki, Maliheh Izadi, Sarah D’Angelo, Sarah Inman, Anielle Andrade, and André van der Hoek. 2025. The Impact of Generative AI on Creativity in Software Development: A Research Agenda. *ACM Transactions on Software Engineering and Methodology* 34, 5, Article 133 (2025). doi:10.1145/3708523
- [53] Ivar Jacobson and Bertrand Meyer. 2009. Methods need theory. *Dr. Dobbs’s Journal* 6 (2009).
- [54] Slinger Jansen, Sjaak Brinkkemper, and Anthony Finkelstein. 2007. Providing Transparency in the Business of Software: A Modeling Technique for Software Supply Networks. In *Software Product Management (IFIP International Federation for Information Processing, Vol. 243)*. Springer, 677–686. doi:10.1007/978-0-387-73798-0_73
- [55] Slinger Jansen, Anthony Finkelstein, and Sjaak Brinkkemper. 2009. A Sense of Community: A Research Agenda for Software Ecosystems. In *31st International Conference on Software Engineering – Companion Volume*. 187–190. doi:10.1109/ICSE-COMPANION.2009.5070978
- [56] Wilhelm Johannsen. 1909. *Elemente der exakten Erblichkeitslehre*. Gustav Fischer, Jena.
- [57] Gary Johns. 2006. The Essential Impact of Context on Organizational Behavior. *Academy of Management Review* 31, 2 (2006), 386–408. doi:10.5465/amr.2006.20208687
- [58] Pontus Johnson and Mathias Ekstedt. 2016. The Tarpit – A general theory of software engineering. *Information and Software Technology* 70 (Feb. 2016), 181–203. doi:10.1016/j.infsof.2015.06.001
- [59] Pontus Johnson, Mathias Ekstedt, and Ivar Jacobson. 2012. Where’s the theory for software engineering? *IEEE Software* 29, 5 (2012), 96–96. doi:10.1109/MS.2012.127
- [60] Lawrence G. Jones and Albert Soule. 2002. *Software Process Improvement and Product Line Practice: CMMI and the Framework for Software Product Line Practice*. Technical Report CMU/SEI-2002-TN-012. Software Engineering Institute, Carnegie Mellon University. doi:10.1184/R1/6584105.v1
- [61] Kyo C. Kang, Sholom G. Cohen, James A. Hess, William E. Novak, and A. Spencer Peterson. 1990. *Feature-Oriented Domain Analysis (FODA) Feasibility Study*. Technical Report CMU/SEI-90-TR-21. Software Engineering Institute, Carnegie Mellon University.
- [62] Barbara A. Kitchenham, Tore Dybå, and Magne Jørgensen. 2004. Evidence-Based Software Engineering. In *Proceedings of the 26th International Conference on Software Engineering*. IEEE Computer Society, 273–281. doi:10.1109/ICSE.2004.1317449
- [63] Boyana Konforti. 2000. Inborn error of metabolism. *Nature Structural Biology* 7, 7 (2000), 535–536. doi:10.1038/76739
- [64] Manny M Lehman. 1984. Program evolution. *Information Processing & Management* 20, 1-2 (1984), 19–36. doi:10.1016/0306-4573(84)90037-2
- [65] Dorothy E. Leidner and Olgerta Tona. 2021. A Thought-Gear Model of Theorizing from Literature. *Journal of the Association for Information Systems* 22, 4 (2021), 874–892. doi:10.17705/1jais.00683

- [66] Leonardo Leite, Nelson Lago, Claudia Melo, Fabio Kon, and Paulo Meirelles. 2023. A Theory of Organizational Structures for Development and Infrastructure Professionals. *IEEE Transactions on Software Engineering* 49, 4 (2023), 1898–1911. doi:10.1109/TSE.2022.3199169
- [67] Mengzhen Li and Mattia Fazzini. 2026. Automatically Resolving Mismatched Invocations on Test Doubles in Test Suites. In *Proceedings of the 2026 IEEE International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 129–140. doi:10.1109/ICST69053.2026.00027
- [68] Deborah J. MacInnis. 2011. A Framework for Conceptual Contributions in Marketing. *Journal of Marketing* 75, 4 (2011), 136–154. doi:10.1509/jmkg.75.4.136
- [69] Tim Mackinnon, Steve Freeman, and Philip Craig. 2001. Endo-Testing: Unit Testing with Mock Objects. In *Extreme Programming Examined*, Giancarlo Succi and Michele Marchesi (Eds.). Addison-Wesley, Boston, MA, 287–301. Originally presented at XP2000.
- [70] William J. McGuire. 1997. Creative Hypothesis Generating in Psychology: Some Useful Heuristics. *Annual Review of Psychology* 48 (1997), 1–30. doi:10.1146/annurev.psych.48.1.1
- [71] Daniel Mendez, Paris Avgeriou, Marcos Kalinowski, and Nauman Bin Ali (Eds.). 2024. *Handbook on teaching empirical software engineering*. Springer. doi:10.1007/978-3-031-71769-7
- [72] Gerard Meszaros. 2004. A Pattern Language for XUnit Testing of Indirect Inputs and Outputs. In *Proceedings of the 11th Conference on Pattern Languages of Programs (PloP 2004)*. The Hillside Group, Monticello, IL, USA. No DOI assigned. https://hillside.net/plop/2004/papers/gmeszaros1/PloP2004_gmeszaros1_0.htm
- [73] Bertrand Meyer. 1992. Applying “Design by Contract”. *IEEE Computer* 25, 10 (1992), 40–51. doi:10.1109/2.161279
- [74] Audris Mockus, Roy T. Fielding, and James D. Herbsleb. 2002. Two Case Studies of Open Source Software Development: Apache and Mozilla. *ACM Transactions on Software Engineering and Methodology* 11, 3 (2002), 309–346. doi:10.1145/567793.567795
- [75] Shaikh Mostafa and Xiaoyin Wang. 2014. An Empirical Study on the Usage of Mocking Frameworks in Software Testing. In *2014 14th International Conference on Quality Software*. IEEE, 127–132. doi:10.1109/QSIC.2014.19
- [76] Jagadeesh Nandigam, Venkat N. Gudivada, Abdelwahab Hamou-Lhadj, and Yonglei Tao. 2009. Interface-Based Object-Oriented Design with Mock Objects. In *2009 Sixth International Conference on Information Technology: New Generations*. IEEE, 713–718. doi:10.1109/ITNG.2009.268
- [77] Isaac Newton. 1687. *Philosophiae Naturalis Principia Mathematica*. Joseph Streater, London.
- [78] Anh Nguyen-Duc, Beatriz Cabrero-Daniel, Adam Przybylek, Chetan Arora, Dron Khanna, Tomas Herda, Usman Rafiq, Jorge Melegati, Eduardo Guerra, Kai-Kristian Kemell, Mika Saari, Zheyang Zhang, Huy Le, Tho Quan, and Pekka Abrahamsson. 2025. Generative Artificial Intelligence for Software Engineering—A Research Agenda. *Software: Practice and Experience* 55, 11 (2025), 1806–1843. doi:10.1002/spe.70005
- [79] Delano Oliveira, Reynold Bruno, Fernanda Madeiral, and Fernando Castor. 2020. Evaluating Code Readability and Legibility: An Examination of Human-Centric Studies. In *Proceedings of the 36th IEEE International Conference on Software Maintenance and Evolution*. 348–359. doi:10.1109/ICSME46990.2020.00041
- [80] A. Ortiz de Guinea and Richard T. Watson. 2025. Practical Techniques for Theorizing from Literature Reviews. *Business & Information Systems Engineering* 67, 6 (Oct. 2025), 939–955. doi:10.1007/s12599-024-00900-8
- [81] Liakat Ali Parapia. 2008. History of bloodletting by phlebotomy. *British Journal of Haematology* 143, 4 (Oct. 2008), 490–495. doi:10.1111/j.1365-2141.2008.07361.x
- [82] David L. Parnas. 1972. On the Criteria To Be Used in Decomposing Systems into Modules. *Commun. ACM* 15, 12 (1972), 1053–1058. doi:10.1145/361598.361623
- [83] Gustavo Pereira and André Hora. 2020. Assessing Mock Classes: An Empirical Study. In *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 453–463. doi:10.1109/ICSME46990.2020.00050
- [84] Mauro Pezzè, Silvia Abrahão, Birgit Penzenstadler, Denys Poshyvanyk, Abhik Roychoudhury, and Tao Yue. 2025. A 2030 Roadmap for Software Engineering. *ACM Transactions on Software Engineering and Methodology* 34, 5 (May 2025), 1–55. doi:10.1145/3731559
- [85] Anna Piro, Giuseppe Tagarelli, Paolo Lagonia, Aldo Quattrone, and Antonio Tagarelli. 2010. Archibald Edward Garrod and alcaptonuria: “Inborn errors of metabolism” revisited. *Genetics in Medicine* 12, 8 (Aug. 2010), 475–476. doi:10.1097/gim.0b013e3181e68843
- [86] Klaus Pohl, Günter Böckle, and Frank van der Linden. 2005. *Software Product Line Engineering: Foundations, Principles and Techniques*. Springer.
- [87] Marshall Scott Poole and Andrew H. Van de Ven. 1989. Using Paradox to Build Management and Organization Theories. *Academy of Management Review* 14, 4 (1989), 562–578. doi:10.5465/amr.1989.4308389
- [88] Aniket Potdar and Emad Shihab. 2014. An exploratory study on self-admitted technical debt. In *2014 IEEE International Conference on Software Maintenance and Evolution*. IEEE, 91–100. doi:10.1109/ICSME.2014.31
- [89] Paul Ralph. 2015. The sensemaking-coevolution-implementation theory of software design. *Science of Computer Programming* 101 (2015), 21–41. doi:10.1016/j.scico.2014.11.007

- [90] Paul Ralph. 2019. Toward methodological guidelines for process theories and taxonomies in software engineering. *IEEE Transactions on Software Engineering* 45, 7 (2019), 712–735. doi:10.1109/TSE.2018.2796554
- [91] Suzanne Rivard. 2021. Theory Building Is Neither an Art nor a Science. It Is a Craft. *Journal of Information Technology* 36, 3 (2021), 316–328. doi:10.1177/0268396220911938
- [92] Knut H Rolland, Brian Fitzgerald, Torgeir Dingsøy, and Klaas-Jan Stol. 2023. Acrobats and safety nets: problematizing large-scale agile software development. *ACM Transactions on Software Engineering and Methodology* 33, 2, Article 33 (2023), 45 pages. doi:10.1145/3617169
- [93] Per Runeson, Martin Höst, Austen Rainer, and Björn Regnell. 2012. *Case Study Research in Software Engineering: Guidelines and Examples*. Wiley. doi:10.1002/9781118181034
- [94] Ernest Rutherford. 1911. The Scattering of α and β Particles by Matter and the Structure of the Atom. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science* 21, 125 (1911), 669–688. doi:10.1080/14786440508637080
- [95] David Saff and Michael D. Ernst. 2004. Automatic Mock Object Creation for Test Factoring. In *Proceedings of the 2004 ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools and Engineering (PASTE '04)*. Association for Computing Machinery, New York, NY, USA, 49–51. doi:10.1145/996821.996838
- [96] Hesam Samimi, Rebecca Hicks, Ari Fogel, and Todd D. Millstein. 2013. Declarative Mocking. In *Proceedings of the 2013 International Symposium on Software Testing and Analysis*. Association for Computing Machinery, New York, NY, USA, 246–256. doi:10.1145/2483760.2483790
- [97] Jörgen Sandberg and Mats Alvesson. 2011. Ways of Constructing Research Questions: Gap-Spotting or Problematisation? *Organization* 18, 1 (2011), 23–44. doi:10.1177/1350508410372151
- [98] Giovanni Sartori. 1970. Concept Misformation in Comparative Politics. *American Political Science Review* 64, 4 (1970), 1033–1053. doi:10.2307/1958356
- [99] Carolyn B. Seaman. 1999. Qualitative Methods in Empirical Studies of Software Engineering. *IEEE Transactions on Software Engineering* 25, 4 (1999), 557–572. doi:10.1109/32.799955
- [100] Agnia Sergeyuk, Yaroslav Golubev, Timofey Bryksin, and Iftekhar Ahmed. 2025. Using AI-based coding assistants in practice: State of affairs, perceptions, and ways forward. *Information and Software Technology* 178 (2025), 107610. doi:10.1016/j.infsof.2024.107610
- [101] Claude E. Shannon. 1938. A Symbolic Analysis of Relay and Switching Circuits. *Transactions of the American Institute of Electrical Engineers* 57, 12 (1938), 713–723. doi:10.1109/EE.1938.6431064
- [102] Dean A. Shepherd and Roy Suddaby. 2017. Theory Building: A Review and Integration. *Journal of Management* 43, 1 (2017), 59–86. doi:10.1177/0149206316647102
- [103] Pamela J. Shoemaker, James W. Tankard, and Dominic L. Lasorsa. 2004. *How to Build Social Science Theories*. SAGE Publications, Thousand Oaks, CA.
- [104] Forrest Shull, Janice Singer, and Dag I. K. Sjøberg (Eds.). 2008. *Guide to Advanced Empirical Software Engineering*. Springer, London. doi:10.1007/978-1-84800-044-5
- [105] Dag IK Sjøberg, Tore Dybå, Bente CD Anda, and Jo E Hannay. 2008. Building theories in software engineering. In *Guide to advanced empirical software engineering*. Springer, 312–336. doi:10.1007/978-1-84800-044-5_12
- [106] Fritz Solms and Linda Marshall. 2016. Contract-Based Mocking for Services-Oriented Development. In *Proceedings of the Annual Conference of the South African Institute of Computer Scientists and Information Technologists (SAICSIT '16)*. Association for Computing Machinery, New York, NY, USA, Article 40, 8 pages. doi:10.1145/2987491.2987534
- [107] Davide Spadini, Mauricio Aniche, Magiel Bruntink, and Alberto Bacchelli. 2019. Mock Objects for Testing Java Systems: Why and How Developers Use Them, and How They Evolve. *Empirical Software Engineering* 24, 3 (2019), 1461–1498. doi:10.1007/s10664-018-9663-0
- [108] Diomidis Spinellis and Clemens Szyperski. 2004. How is open source affecting software development? *IEEE Software* 21, 1 (2004), 28–33. doi:10.1109/MS.2004.1259204
- [109] Tor Sporse, Torgeir Dingsøy, and Klaas-Jan Stol. 2026. User stories as boundary objects in agile requirements engineering: A theoretical literature review. *Journal of Systems and Software* 233 (March 2026), 112693. doi:10.1016/j.jss.2025.112693
- [110] Richard Staley. 2008. Worldviews and Physicists' Experience of Disciplinary Change: On the Uses of 'Classical' Physics. *Studies in History and Philosophy of Science Part A* 39, 3 (2008), 298–311. doi:10.1016/j.shpsa.2008.06.002
- [111] Klaas-Jan Stol. 2024. Teaching theorizing in software engineering research. In *Handbook on Teaching Empirical Software Engineering*. Springer, 31–69. doi:10.1007/978-3-031-71769-7_3
- [112] Klaas-Jan Stol. 2026. A Show About Nothing: Why Software Engineering Researchers Need to Change the Conversation. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering: Future of Software Engineering*. Accepted/In press.
- [113] Klaas-Jan Stol, Paris Avgeriou, Muhammad Ali Babar, Yan Lucas, and Brian Fitzgerald. 2014. Key factors for adopting inner source. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 23, 2 (2014), 1–35. doi:10.1145/25

- 33685
- [114] Klaas-Jan Stol and Brian Fitzgerald. 2014. Two's company, three's a crowd: a case study of crowdsourcing software development. In *Proceedings of the 36th international conference on software engineering*. 187–198. doi:10.1145/2568225.2568249
 - [115] Klaas-Jan Stol and Brian Fitzgerald. 2015. Theory-Oriented Software Engineering. *Science of Computer Programming* 101 (2015), 79–98. doi:10.1016/j.scico.2014.11.010
 - [116] Klaas-Jan Stol and Brian Fitzgerald. 2018. The ABC of Software Engineering Research. *ACM Transactions on Software Engineering and Methodology* 27, 3, Article 11 (2018), 51 pages. doi:10.1145/3241743
 - [117] Klaas-Jan Stol, Michael Goedicke, and Ivar Jacobson. 2016. Introduction to the special section—General Theories of Software Engineering: New advances and implications for research. *Information and Software Technology* 70 (2016), 176–180. doi:10.1016/j.infsof.2015.07.010
 - [118] Klaas-Jan Stol, Paul Ralph, and Brian Fitzgerald. 2016. Grounded Theory in Software Engineering Research: A Critical Review and Guidelines. In *Proceedings of the 38th International Conference on Software Engineering*. 120–131. doi:10.1145/2884781.2884833
 - [119] Klaas-Jan Stol, Mario Schaarschmidt, and Shelly Goldblit. 2022. Gamification in software engineering: the mediating role of developer engagement and job satisfaction. *Empirical Software Engineering* 27, 2 (2022), 35. doi:10.1007/s10664-021-10062-w
 - [120] Roy Suddaby. 2010. Editor's Comments: Construct Clarity in Theories of Management and Organization. *Academy of Management Review* 35, 3 (2010), 346–357. doi:10.5465/amr.35.3.zok346
 - [121] Mikael Svahnberg, Jilles van Gurp, and Jan Bosch. 2005. A Taxonomy of Variability Realization Techniques. *Software: Practice and Experience* 35, 8 (2005), 705–754. doi:10.1002/spe.652
 - [122] Richard Swedberg. 2014. *The Art of Social Theory*. Princeton University Press, Princeton, NJ. doi:10.23943/princeton/9780691155227.001.0001
 - [123] Dave Thomas and Andy Hunt. 2002. Mock Objects. *IEEE Software* 19, 3 (2002), 22–24. doi:10.1109/MS.2002.1003449
 - [124] Bianca Trinkenreich, Fabio Santos, and Klaas-Jan Stol. 2024. Predicting attrition among software professionals: Antecedents and consequences of burnout and engagement. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024), 45 pages. doi:10.1145/3691629
 - [125] Eric W. K. Tsang and Florian Ellsaesser. 2011. How Contrastive Explanation Facilitates Theory Building. *Academy of Management Review* 36, 2 (2011), 404–419. doi:10.5465/amr.2009.0153
 - [126] Andrew H. Van de Ven and Marshall Scott Poole. 1995. Explaining Development and Change in Organizations. *Academy of Management Review* 20, 3 (1995), 510–540. doi:10.5465/amr.1995.9508080329
 - [127] John G. Wacker. 1998. A Definition of Theory: Research Guidelines for Different Theory-Building Research Methods in Operations Management. *Journal of Operations Management* 16, 4 (1998), 361–385. doi:10.1016/S0272-6963(98)00019-9
 - [128] Karl E. Weick. 1989. Theory Construction as Disciplined Imagination. *Academy of Management Review* 14, 4 (1989), 516–531. doi:10.5465/amr.1989.4308376
 - [129] Karl E. Weick. 1995. What Theory Is Not, Theorizing Is. *Administrative Science Quarterly* 40, 3 (1995), 385–390. doi:10.2307/2393789
 - [130] Colin Werner, Laura Okpara, Klaas-Jan Stol, and Daniela Damian. 2026. Maintaining shared understanding of non-functional requirements in small companies using continuous software engineering. *Empirical Software Engineering* 31, 2 (2026), 40. doi:10.1007/s10664-025-10765-4
 - [131] John B. West. 2014. Joseph Black, carbon dioxide, latent heat, and the beginnings of the discovery of the respiratory gases. *American Journal of Physiology-Lung Cellular and Molecular Physiology* 306, 12 (2014), L1057–L1063. doi:10.1152/ajplung.00020.2014
 - [132] David A. Whetten. 1989. What Constitutes a Theoretical Contribution? *Academy of Management Review* 14, 4 (1989), 490–495. doi:10.5465/amr.1989.4308371
 - [133] David A. Whetten. 2009. An Examination of the Interface between Context and Theory Applied to the Study of Chinese Organizations. *Management and Organization Review* 5, 1 (2009), 29–55. doi:10.1111/j.1740-8784.2008.00132.x
 - [134] Roel Wieringa and Maya Daneva. 2015. Six strategies for generalizing software engineering theories. *Science of Computer Programming* 101 (April 2015), 136–152. doi:10.1016/j.scico.2014.11.013
 - [135] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. 2012. *Experimentation in Software Engineering*. Springer, Berlin, Heidelberg. doi:10.1007/978-3-642-29044-2
 - [136] Claes Wohlin, Darja Šmite, and Nils Brede Moe. 2015. A general theory of software engineering: Balancing human, social and organizational capitals. *Journal of Systems and Software* 109 (Nov. 2015), 229–242. doi:10.1016/j.jss.2015.08.009
 - [137] Lu Xiao, Gengwu Zhao, Xiao Wang, Keye Li, Erick Lim, Chenhao Wei, Tingting Yu, and Xiaoyin Wang. 2024. An Empirical Study on the Usage of Mocking Frameworks in Apache Software Foundation. *Empirical Software Engineering* 29, 2, Article 39 (2024). doi:10.1007/s10664-023-10410-y

- [138] Shaker A. Zahra. 2007. Contextualizing Theory Building in Entrepreneurship Research. *Journal of Business Venturing* 22, 3 (2007), 443–452. doi:10.1016/j.jbusvent.2006.04.007
- [139] Hengcheng Zhu, Valerio Terragni, Lili Wei, Shing-Chi Cheung, Jiarong Wu, and Yepang Liu. 2025. Understanding and Characterizing Mock Assertions in Unit Tests. *Proceedings of the ACM on Software Engineering* 2, FSE, Article FSE026 (2025), 554–575 pages. doi:10.1145/3715741
- [140] Hengcheng Zhu, Lili Wei, Valerio Terragni, Yepang Liu, Shing-Chi Cheung, Jiarong Wu, Qin Sheng, Bing Zhang, and Lihong Song. 2024. StubCoder: Automated Generation and Repair of Stub Code for Mock Objects. *ACM Transactions on Software Engineering and Methodology* 33, 1, Article 16 (2024), 31 pages. doi:10.1145/3617171

A Synthesis of Prior Theorizing Guidelines

The XYZ framework emerged from a consultation of prior literature on theorizing, as well as the author’s reflections on his theorizing experience. Table 7 provides an overview of the author’s retrospective and implicit use of these operations; this is an additional set of examples of these operations, in addition to those presented in the remainder of the paper.

Before starting the synthesis, it was important to define what constitutes a theoretical operation; given the lack of a general definition of “theory,” and the many forms that theory can take, there is no single theory development process. Indeed, many different guidelines exist, and it is easy to associate a wide range of activities with theorizing. For example: one can describe, compare, contrast, investigate, justify, hypothesize, and so on. However, many of these activities are either composites, i.e., actions that do multiple things, or they do not effect a theoretical transformation.

A composite operation comprises multiple indivisible operations. For example, to hypothesize does multiple things; for variance theories (as opposed to process theories), (a) it links together at least two variables, and (b) it implies an explanation of *why* they are linked. Perhaps more importantly, hypotheses can do different things: they can offer an explanation, they can suggest certain conditions under which a relationship holds, they could challenge an existing hypothesis, or they could instantiate a general hypothesis in a specific setting. For these reasons, hypothesize is not deemed an elemental operation by itself, and is better thought of as a theorizing tactic (see Table 1).

Other activities do not effect a theoretical transformation, and therefore they do not make a theoretical contribution; for example, to describe is not a theoretical operation. That is to say, while there can be great value in describing a phenomenon, by itself it does not contribute to theory. The same is true for comparing, investigating, and so on. Only when the scholar identifies and names a new phenomenon or clarifies its differences with other phenomena, for example, do these activities add theoretical contributions. Thus, during the synthesis process the focus was on identifying operations that effect some theoretical change.

Scholars from several scientific disciplines have published a variety of guidelines and opinions on the nature of theory and the theory development process. Starting with a set of articles and editorials that the author had consulted over the years, an initial review identified several operations. Personal reflections by the author also led to the identification of some operations used in his prior work; for example, Alvesson and Sandberg’s problematization methodology [2, 92], which was captured within the framework as the ‘challenge’ operation. A further literature identified additional sources. Much of this guidance came from disciplines that have prioritized the role of theory, including management and information systems. All of these were read in detail and relevant fragments of text were highlighted and annotated. This quickly led to a stable set of eight operations.

A second step was to determine the relationships among them; because eight operations were identified, the circumplex structure was deemed an appropriate starting point to lay out these operations, akin to how the ABC framework that the author has previously adopted from McGrath [116], which lays out eight research strategies. The positioning of the eight operations emerged

Table 7. Examples of theorizing operations in the author's prior research

Operation	Example	Theoretical transformation
Name	We observed that systems adoption is not a single event, but can recur as systems are continuously changed and redeployed. We labeled this phenomenon “continuous adoption” [4].	The concept shifts attention from adoption as a one-off event to adoption as a recurring process. Once named, continuous adoption becomes an object about which questions can be asked concerning its dimensions, causes, consequences, and relationship to CSE and established adoption theory.
Abstract	A multi-disciplinary literature review led to a higher-level concept “Continuous*”. The concept captures a common underlying logic across a number of continuous activities, including shorter feedback cycles, increased frequency, smaller batches, and tighter integration of activities [33].	Previously separate developments such as CI/CD, DevOps, and related continuous activities are represented as manifestations of a more general phenomenon. This abstraction enables questions about properties shared across continuous practices and about how continuity might extend to other parts of the software lifecycle.
Explain	A study of gamification in software engineering developed and tested a theoretical model to explain why developers participate in gamified activities and how developer engagement is related to job satisfaction [119].	The study provides an explanation of how job satisfaction arises, with developer engagement playing a central explanatory role. This makes the mechanisms and intermediate relationships themselves available for subsequent testing, refinement, and qualification.
Connect	A study of crowdsourced software development related the emerging crowdsourcing model to established software engineering concerns [114].	Raises a new set of questions originating in the general crowdsourcing literature that can now be asked of crowdsourcing in an SE context.
Bound	Identified nine “key factors” that should be considered when assessing whether Inner Source is appropriate for an organization [113].	The study specifies organizational conditions under which inner source adoption is viable. This shifts the focus from “inner source is beneficial for organizations in general” to “inner source offers benefits under this set of specific circumstances.”
Challenge	Questioning assumptions underpinning the literature on large-scale agile development, this study identifies three such assumptions and demonstrates through a longitudinal case study that they are untenable [92].	The study offers an alternative theoretical perspective in which agile and plan-driven practices can be mutually enabling.
Instantiate	A theoretical review of research on user stories applied Boundary Object Theory to understand how user stories operate across knowledge boundaries in agile requirements engineering, resulting in five theoretical propositions [109].	The instantiation of boundary object theory makes visible context-specific mechanisms, such as how user stories support knowledge transfer across syntactic, semantic, and pragmatic boundaries, and generates new propositions and questions about when user stories succeed or fail as boundary objects.
Distinguish	A critical review emphasizes that there is not a single version of Grounded Theory (GT), and that researchers should neither claim to use GT when employing only selected techniques nor ignore the principal variants of the methodology [118].	Treating the variants separately, rather than collapsing them into an undifferentiated “GT”, makes it possible to ask whether a study's procedures and claims are coherent with the particular variant it invokes and to identify “method slurring” where incompatible or partial practices are combined.

through the juxtaposition of all pairs of operations, so as to ‘fit’ them together, in a similar way as one would make a jigsaw puzzle.